

Article

Discrete Physics-Informed Training for Projection-Based Reduced-Order Models with Neural Networks

Nicolas Sibuet ¹, Sebastian Ares de Parga ^{1,2,*} , Jose Raul Bravo ^{1,2}  and Riccardo Rossi ^{1,2}

¹ Department of Civil and Environmental Engineering (DECA), Universitat Politècnica de Catalunya, 08034 Barcelona, Spain; nicolas.sibuet@upc.edu (N.S.); jose.raul.bravo@upc.edu (J.R.B.); riccardo.rossi@upc.edu (R.R.)

² Centre Internacional de Mètodes Numèrics en Enginyeria (CIMNE), 08034 Barcelona, Spain

* Correspondence: sebastian.ares@upc.edu

Abstract: This paper presents a physics-informed training framework for projection-based Reduced-Order Models (ROMs). We extend the original PROM-ANN architecture by complementing snapshot-based training with a FEM-based, discrete physics-informed residual loss, bridging the gap between traditional projection-based ROMs and physics-informed neural networks (PINNs). Unlike conventional PINNs that rely on analytical PDEs, our approach leverages FEM residuals to guide the learning of the ROM approximation manifold. Our key contributions include the following: (1) a parameter-agnostic, discrete residual loss applicable to nonlinear problems, (2) an architectural modification to PROM-ANN improving accuracy for fast-decaying singular values, and (3) an empirical study on the proposed physics-informed training process for ROMs. The method is demonstrated on a nonlinear hyperelasticity problem, simulating a rubber cantilever under multi-axial loads. The main accomplishment in regards to the proposed residual-based loss is its applicability on nonlinear problems by interfacing with FEM software while maintaining reasonable training times. The modified PROM-ANN outperforms POD by orders of magnitude in snapshot reconstruction accuracy, while the original formulation is not able to learn a proper mapping for this use case. Finally, the application of physics-informed training in ANN-PROM modestly narrows the gap between data reconstruction and ROM accuracy; however, it highlights the untapped potential of the proposed residual-driven optimization for future ROM development. This work underscores the critical role of FEM residuals in ROM construction and calls for further exploration on architectures beyond PROM-ANN.



Academic Editor: Oscar Montiel Ross

Received: 3 April 2025

Revised: 15 May 2025

Accepted: 16 May 2025

Published: 20 May 2025

Citation: Sibuet, N.; Ares de Parga, S.;

Bravo, J.R.; Rossi, R. Discrete

Physics-Informed Training for

Projection-Based Reduced-Order

Models with Neural Networks.

Axioms **2025**, *14*, 385. [https://doi.org/](https://doi.org/10.3390/axioms14050385)

10.3390/axioms14050385

Copyright: © 2025 by the authors.

Licensee MDPI, Basel, Switzerland.

This article is an open access article

distributed under the terms and

conditions of the Creative Commons

Attribution (CC BY) license

([https://creativecommons.org/](https://creativecommons.org/licenses/by/4.0/)

[licenses/by/4.0/](https://creativecommons.org/licenses/by/4.0/)).

Keywords: reduced-order model (ROM); physics-informed neural networks (PINNs); artificial neural network (ANN); projection-based model reduction; proper orthogonal decomposition (POD)

MSC: 65M60; 68T07

1. Introduction

In recent years, the development of increasingly sophisticated high-fidelity models (HFMs) has become crucial for simulating complex physical phenomena. One of the main computational challenges in HFMs is the need for very fine spatio-temporal resolutions, which results in extremely high-dimensional problems that can take weeks to solve, even on numerous parallelized computing cores. Therefore, it is vital to create methods that significantly reduce the computational time and resources required to enable the application

of these models in time-sensitive scenarios, including real-time control systems, simulation-driven optimization and digital twins [1–3].

To address these computational challenges, projection-based reduced-order models (ROMs) have emerged as a powerful strategy, allowing for rapid simulation by approximating the dynamics of the high-dimensional system within a lower-dimensional latent space [4]. ROMs can be broadly classified as intrusive or non-intrusive, depending on how the reduced equations are derived. Intrusive ROMs project the governing equations directly onto a reduced space, typically via Galerkin projection [5], Least-Squares Petrov–Galerkin (LSPG) projection [6–9], or related Petrov–Galerkin formulations [10]. In contrast, non-intrusive ROMs avoid direct equation manipulation, instead employing supervised-learning methods like artificial neural networks (ANNs) or Gaussian processes to map parameters directly to reduced solutions, thereby fully decoupling the online computation from the original high-fidelity solver [11–15].

Intrusive projection-based ROMs typically involve an offline–online decomposition. In the offline stage, a latent space is computed from a collection of high-fidelity solutions sampled over a defined parametric domain. Proper Orthogonal Decomposition (POD) is a common approach, relying on Singular Value Decomposition (SVD) to obtain an optimal linear basis that efficiently captures the dominant dynamics [16–18]. The online stage then involves solving a reduced optimization problem within this latent space, resulting in dramatically reduced computational costs compared to the original HFM. Such strategies have demonstrated effectiveness in parametric linear problems [19,20] and nonlinear problems with moderate complexities [21,22].

Non-intrusive ROMs have attracted attention for scenarios where intrusive access to HFM operators is impractical or unavailable. Early successes include Gaussian processes and ANN regression models for aeroacoustic and structural problems [13,15], as well as neural networks for predicting flow coefficients in combustion dynamics [11,12]. Recent extensions leverage convolutional autoencoders for complex spatio-temporal predictions [23] and generalized data-driven frameworks for nonlinear PDEs [14]. Later work with graph neural networks [24] has aimed at bypassing the need for structured meshes that convolutional autoencoders have. Despite their advantages in computational efficiency, these methods require large training datasets and their scale is still directly tied to the HFM’s dimensions.

Nevertheless, intrusive linear ROMs, particularly those using POD–Galerkin formulations, have been noted to exhibit accuracy degradation and stability issues in highly nonlinear or convection-dominated regimes. For example, spurious mode growth was reported in compressible flows [21], stability issues have been examined from dynamical systems perspectives [25], and convergence difficulties have been highlighted in reactive flow scenarios [26]. These limitations have been attributed to the slow decay of the Kolmogorov n -width [27], fundamentally restricting the representational power of linear subspaces.

Consequently, a growing body of research has focused on developing nonlinear ROM strategies to overcome these fundamental expressivity barriers. Within intrusive ROMs, the formal theory for generic nonlinear projection-based ROM was introduced in [28], which was first demonstrated using a convolutional neural network to define a nonlinear mapping between the full and latent spaces. Despite several limitations—namely, limited scalability and the need for structured meshes—this architecture paved the way for further nonlinear architectures. These include methods based on quadratic manifolds [29], piecewise linear manifolds via local POD approaches [9,30,31], and the PROM-ANN architecture from [32,33], the latter essentially being a nonlinear approximation of POD using dense neural networks.

Another approach to deal with nonlinearizable dynamics is the use of spectral submanifolds [34,35], which are more mathematically sound than data-based ones. These type of nonlinear manifolds can be found under certain non-resonance conditions [36], and their direct computation requires explicit knowledge of nonlinear coefficients in the equations of motion. Recent developments in [34] present a non-intrusive and data-free method to obtain these spectral submanifolds, but are currently limited to mechanical systems with cubic order nonlinearities.

In this paper, we propose a novel residual-informed training approach for constructing nonlinear ROM operators. While existing projection-based ROMs construct their projection operators based exclusively on learning the reconstruction of solution snapshots, we propose incorporating the discrete residual of the FEM-based HFM—the same residual used during online ROM projection—into the training of these operators. The reason for this is that since the residual is the quantity to be optimized during ROM inference, its more accurate representation will improve the solutions of the ROMs, which is much needed.

Our methodology is inspired by Physics-Informed Neural Networks (PINNs) [37] and their variants, e.g., [38,39]. PINNs are models based on neural networks that learn the solution function to a system of partial differential equations (PDEs) in their continuous form. This is achieved by embedding the governing PDEs directly into the loss function and taking advantage of the automatic differentiation capabilities of common machine learning libraries. However, their application in engineering workflows can be hindered by their difficulties to cope with irregular or discontinuous domains, complex PDEs due to spectral bias (which hinders their ability to handle high-frequency terms [40]), discontinuities (such as shock waves [41]), and plasticity (requiring significant effort to develop workarounds for these issues [42]).

In contrast, the Finite Element Method (FEM) [43], along with other HFMs like Finite Differences or Finite Volumes Methods, remains the gold standard for solving complex physical behaviors, particularly in engineering applications. This has led to recent efforts to bridge FEM and PINNs. For example, Refs. [44,45] proposed discrete PINNs that use FEM to compute residuals and their derivatives for backpropagation in linear problems. These approaches predict nodal values using neural networks that take simulation parameters as the only input. Meanwhile, the spatial discretization and boundary conditions are implicitly integrated via the FEM residual. While [45] focuses exclusively on structural mechanics, Ref. [44] generalizes the approach but only for linear PDEs. Another intermediate approach is presented in [46], which introduces a physics-informed neural operator inference [47] framework that takes a discretized, variational form of the PDE as the training loss. This variational form is based on the energy of the system and is closely related to the FEM one, although it is not generalizable to the same degree. They remark how traditional PINNs only enforce the strong form of the PDE on the collocation points, resulting in a lack of global smoothness, while a variational approach ensures this implicitly.

Other semi-intrusive strategies have been proposed that embed reduced-order residual information into neural networks to enhance generalization and enforce physical consistency in parametric regimes. We refer to these as semi-intrusive methods, since they require partial access to governing equations or their projections during training—such as reduced residuals—but retain a decoupled, non-intrusive structure at inference time. In [48], a Physics-Reinforced Neural Network (PRNN) was introduced, which minimizes a hybrid loss composed of reduced residuals and projection data in the latent space. Subsequently, Ref. [49] combined POD-Galerkin ROMs with a neural network trained on both data and the residual of the reduced Navier–Stokes equations, enabling the same architecture to address forward prediction and inverse parameter-identification tasks. Moreover, Ref. [50] enhanced POD–DL-ROMs by incorporating a strong-form PDE residual into the train-

ing process and adopting a pre-train plus fine-tune strategy that significantly reduces computational cost in nonlinear flow problems.

Beyond reduced-order modeling, broader physics-informed machine learning frameworks such as the Deep Galerkin Method for high-dimensional free-boundary PDEs [51] and PINN-based RANS modeling for turbulent incompressible flows [52] further demonstrate the growing applicability of physics-aware neural architectures to complex nonlinear systems. Related efforts have explored manifold-learning and structure-preserving neural architectures outside traditional ROM contexts, including shallow masked autoencoders for nonlinear mechanical simulations [53], mechanics-informed neural networks for robust constitutive modeling [54], physics-informed architectures for modeling mistuned integrally bladed rotors [55], FE-ROM-informed neural operators for structural dynamics [56], and two-tier deep neural architectures for multiscale nonlinear mechanics [57]. Although these methods primarily remain data-driven, their built-in physics constraints substantially enhance predictive capability. Despite these promising developments, a gap remains in fully leveraging the discrete physics of the HFM directly during the training of projection-based ROMs, particularly within nonlinear manifold approximations.

In this work, we propose a method to incorporate physics information into the ROM nonlinear approximation manifold by using the FEM residual as the training loss. This requires two key components: (1) a flexible ROM architecture capable of accommodating custom loss functions for the definition of the projection operators, and (2) a framework to integrate FEM residuals into the training process. For the architecture, we adapt the PROM-ANN framework from [32,33], which uses neural networks in a scalable manner. We modify this architecture to suit our needs and develop a framework to integrate FEM residuals into a neural network's loss, along with their Jacobians for the backpropagation. Our approach assumes that the user has access to a fully functional FEM software capable of providing these quantities—a reasonable requirement given the need for an HFM to generate training data and perform intrusive ROM.

Our residual-based loss function differs from previous studies in its generality and flexibility. Instead of minimizing the FEM residual norm directly, we compute the difference between the obtained residual and ground truth, thereby learning the behavior of the residual in non-converged conditions. We also provide an optimized implementation for computing the gradient of the residual loss, enabling efficient backpropagation.

To validate our methodology, we apply it to a steady-state structural mechanics problem involving a cantilever modeled by an unstructured mesh. The FEM software of choice is KratosMultiphysics [58], which we adapted for this purpose. Our results show that the modified PROM-ANN architecture, combined with the residual-based loss, achieves slightly but consistently improved accuracy in ROM simulations. We acknowledge that training with the residual loss is computationally expensive compared to the traditional snapshot-based loss. To address this, we propose using residual training only as a fine-tuning step after initial snapshot-based training, significantly reducing overall training time.

While adapting the architecture of PROM-ANN to suit our specific needs, we also identified opportunities to enhance the original design, enabling it to learn effectively across a broader range of scenarios, even without incorporating the residual-based loss. These improvements are presented as additional contributions in this paper.

It should be noted that, although projection-based ROMs have also been employed to accelerate inverse problems [59,60], the present work exclusively addresses forward parametric simulations. Extensions to inverse tasks—such as parameter estimation—would require further developments, such as involving adjoint-based gradients or parameter-to-output mappings, which lie beyond the scope of this study.

In summary, our contributions are threefold:

1. **Physics-Informed Residual-Based Loss:** A residual-based loss function is introduced for training ROMs using discrete FEM residuals. While generally applicable to nonlinear problems and ROM architectures, it is demonstrated here within the PROM-ANN framework. The loss is parameter-agnostic and integrated through a general back-propagation strategy compatible with existing FEM infrastructure.
2. **Enhanced PROM-ANN Architecture:** Modifications are proposed to improve the original PROM-ANN framework, enabling it to handle problems with fast-decaying singular-value spectra. These include scaling strategies that enhance training stability and general applicability.
3. **Quantitative Evaluation of Residual Training:** A comprehensive study is conducted to assess the impact of residual-informed training on both snapshot reconstruction and ROM simulation, demonstrating modest but consistent improvements and laying the groundwork for future refinements.

The application of the physics-informed training in this specific architecture does not yield enough enhancement in terms of accuracy to justify the increase in training time. However, our findings suggest that focusing on residual behavior could unlock further potential in nonlinear ROMs, particularly when combined with architectures specifically designed for this purpose. Possible directions to make the training process faster and to make the effect of the physics-based training more impactful are proposed within this paper.

The rest of the paper is organized as follows. Section 2 provides a review of the main methods that form the basis for this work, i.e., PINNs and projection-based ROM with an emphasis on the original PROM-ANN architecture. In Section 3, we propose a discrete, FEM-based residual loss and develop an adequate implementation strategy for it. Then, Section 4 provides a series of modifications to the original PROM-ANN architecture and loss that make it compatible with problems with fast-decaying singular values. Section 5 effectively merges the developments presented in the two previous sections to enable physics-informed nonlinear ROM. Following this, Section 6 explains the specific FEM problem in which the developments will be tested, as well as the software of choice and the neural network training strategy. Section 7 shows the results of applying the methods developed throughout the paper onto our specific use case. Section 8 further discusses the implications of the results from the previous section and proposes further research directions for future improvements. Finally, Section 9 closes the paper with the most significant conclusions.

2. Background

2.1. Physics-Informed Neural Networks

In this section, we introduce PINNs, highlighting the most relevant aspects to our case. Our focus is on PINNs for forward parametric problems. Comprehensive reviews on the numerous variations of PINNs can be found in, e.g., [38,61].

Consider a physical problem in the following form:

$$\begin{cases} \mathcal{R}(u(z; \mu), \mu) = \mathbf{0} & z \in \Omega \times (0, T], \\ \mathcal{B}(u(z; \mu), z) = g(z, \mu) & z \in \partial\Omega \times (0, T], \\ \mathcal{I}(u(z; \mu), z) = f(z, \mu) & z \in \Omega \times \{0\}, \end{cases} \quad (1)$$

where $\Omega \subset \mathbb{R}^d$ represents the spatial domain with boundary $\partial\Omega$, $u : \Omega \times \mathbb{R}_+ \times \mathbb{R}^p \rightarrow \mathbb{R}^{N_{\text{dof}}}$ is the unknown solution, z is a spatio-temporal coordinates vector, $\mu \in \mathcal{P} \subset \mathbb{R}^p$ is the parameters vector encapsulating geometrical, material, and/or initial and boundary

conditions. $\mathcal{R}(\cdot)$ is the differential operator in residual form encapsulating the physics. $\mathcal{B}(\cdot)$ is an operator indicating boundary conditions, and $\mathcal{I}(\cdot)$ is an operator indicating initial conditions.

In PINNs, one uses a neural network for representing the solution function as

$$\mathbf{u} \approx \mathcal{N}(\mathbf{z}, \boldsymbol{\mu}; \boldsymbol{\Theta}), \quad (2)$$

where $\mathcal{N}$ denotes the neural network, and $\boldsymbol{\Theta}$ encapsulates its learnable parameters. The optimal parameters for the neural network are obtained by minimizing a loss function:

$$\boldsymbol{\Theta}^* = \arg \min_{\boldsymbol{\Theta}} \mathcal{L}(\boldsymbol{\Theta}), \quad (3)$$

where the loss function $\mathcal{L}(\boldsymbol{\Theta})$ can be split into

$$\mathcal{L}(\boldsymbol{\Theta}) = \omega_{\mathcal{R}} \mathcal{L}_{\mathcal{R}}(\boldsymbol{\Theta}) + \omega_d \mathcal{L}_d(\boldsymbol{\Theta}). \quad (4)$$

Here, ω_d and $\omega_{\mathcal{R}}$ are weighting factors that balance the contribution of the data-driven loss $\mathcal{L}_d$ and the residual loss $\mathcal{L}_{\mathcal{R}}$, respectively, to the overall loss function. The data-driven loss $\mathcal{L}_d$ is related to the mismatch between mN_d ground truth data points $\mathbf{u}_{ij}^*$ and the output of the PINN for the same spatio-temporal and parametric coordinates. This data-driven loss $\mathcal{L}_d$ reads

$$\mathcal{L}_d = \text{MSE}_d = \frac{1}{mN_d} \sum_{j=1}^m \sum_{i=1}^{N_d} \left\| \mathcal{N}(\mathbf{z}_i, \boldsymbol{\mu}_j; \boldsymbol{\Theta}) - \mathbf{u}_{ij}^* \right\|^2, \quad (5)$$

where N_d stands for the number of collocation points considered in the spatio-temporal domain. Moreover, m represents the number of points considered in the parametric space $\mathcal{P}$.

The residual loss $\mathcal{L}_{\mathcal{R}}$ relates to the substitution in the residual operator, as defined in Equation (1), of the unknown solution with its neural network approximation as defined in Equation (2), and reads

$$\mathcal{L}_{\mathcal{R}} = \text{MSE}_{\mathcal{R}} = \frac{1}{mN_{\mathcal{R}}} \sum_{j=1}^m \sum_{i=1}^{N_{\mathcal{R}}} \left\| \mathcal{R}(\mathcal{N}(\mathbf{z}_i, \boldsymbol{\mu}_j; \boldsymbol{\Theta}), \boldsymbol{\mu}_j) \right\|^2, \quad (6)$$

where $N_{\mathcal{R}}$ represents the number of residual evaluation points, which might be different from the collocation points used in the data-driven loss. Admittedly, from the substitution of Equation (2) into Equation (1), two more terms could be considered, corresponding to the initial and boundary conditions. However, here we assume that the initial and boundary conditions have been embedded into the PINN, as in [62,63], therefore ensuring their compliance and that they do not enter into the loss function.

The fact that the PINN itself is a differentiable mapping from spatio-temporal coordinates to the corresponding solution means that gradients of the solution and temporal derivatives of it are directly computable within the deep learning software via auto-differentiation. This allows for the internal computation of the loss and its gradient.

2.2. The Full-Order Model

In the initial formulation of our problem, we represented the physics using a continuous residual form, as shown in Equation (1), revisited here for clarity:

$$\mathcal{R}(\mathbf{u}(\mathbf{z}; \boldsymbol{\mu}); \boldsymbol{\mu}) = \mathbf{0} \quad \mathbf{z} \in \Omega \times (0, T]. \quad (7)$$

To adapt this continuous form for computational analysis, we discretize the domain using FEM. By applying FEM discretization to Equation (7) and incorporating initial and

boundary conditions, we arrive at a set of governing equations in the form of a μ -parametric operator $\mathbf{R} : \mathbb{R}^N \times \mathcal{P} \rightarrow \mathbb{R}^N$. This results in the following discrete representation:

$$\mathbf{R}(\mathbf{u}; \mu) = \mathbf{0}, \quad (8)$$

where $\mathbf{u} \in \mathbb{R}^N$ is the FOM solution vector containing the value of the solution on every degree of freedom of the spatial discretization, and $\mu \in \mathcal{P} \subset \mathbb{R}^p$ is the parameters vector encapsulating, e.g., geometrical variations, material properties or boundary conditions.

Given an initial guess of the FOM solution at a step t , symbolically represented as $\mathbf{u}_t^0(\mu)$ (for our current discussion can be either a time step, or a loading step following a predefined trajectory), the actual solution corresponding to the step can be obtained via an iterative method (e.g., Newton's) like:

$$-\mathbf{J}(\mathbf{u}_t^k(\mu); \mu) \delta \mathbf{u}_t^k(\mu) = \mathbf{R}(\mathbf{u}_t^k(\mu); \mu) \quad (9a)$$

$$\mathbf{u}_t^{k+1}(\mu) = \mathbf{u}_t^k(\mu) + \delta \mathbf{u}_t^k(\mu) \quad (9b)$$

where $\mathbf{J} = \frac{\partial \mathbf{R}}{\partial \mathbf{u}}$ is a Jacobian matrix (or an approximation thereof), k is the current nonlinear iteration, and $\delta \mathbf{u}_t^k \in \mathbb{R}^N$ is the solution state increment.

In practice, the FEM discretization can yield millions of degrees of freedom, making each nonlinear iteration prohibitively expensive. This challenge grows acute in scenarios demanding rapid solutions (e.g., design optimization, digital twins). To address this, various surrogate models have emerged, ranging from intrusive methods that directly manipulate the governing equations to non-intrusive black box approaches. This work builds upon an intrusive, projection-based reduced-order model.

2.3. Manifold Projection-Based ROMs

Following the standard procedure introduced in [28] for nonlinear projection-based ROMs (also known as manifold-based ROMs), we begin by approximating the FOM's solution state variable $\mathbf{u}^*(\mu)$ through a general decoder function, defined as follows:

$$\mathbf{u}^*(\mu) \approx \mathbf{u}(\mu) = D_u(\mathbf{q}(\mu)), \quad (10)$$

where $D_u : \mathbb{R}^n \rightarrow \mathbb{R}^N$ maps a reduced solution $\mathbf{q}(\mu)$ in a latent space to a solution $\mathbf{u}(\mu)$ in the full space, and $n \ll N$.

Correspondingly, we introduce its encoder:

$$E_u : \mathbb{R}^N \rightarrow \mathbb{R}^n, \quad \mathbf{u}(\mu) \mapsto \mathbf{q}(\mu). \quad (11)$$

When Equation (10) is substituted into Equation (8), we obtain a residual characterized by the following:

$$\mathbf{R}(\mathbf{u}(\mu); \mu) = \mathbf{R}(D_u(\mathbf{q}(\mu)); \mu). \quad (12)$$

Finding the best reduced solution $\mathbf{q}$ involves solving the following minimization problem:

$$\min_{\mathbf{q}(\mu) \in \mathbb{R}^n} \frac{1}{2} \|\mathbf{R}(D_u(\mathbf{q}(\mu)); \mu)\|_{\mathbf{G}}^2. \quad (13)$$

Particular choices of the norm-defining matrix $\mathbf{G} \in \mathbb{R}^{N \times N}$ lead to different methods. In our particular setting, $\mathbf{G} := \mathbf{J}^{-T}$ leads to the Galerkin method, which we utilize in this work. For the derivation of the Galerkin method and other methods through particular choices of $\mathbf{G}$, the interested reader is directed to [10].

In this way, given an initial guess for the ROM solution, symbolically represented as $\mathbf{u}^0(\mu) \in \mathbb{R}^N$, the ROM solution can be obtained using the Galerkin method by iteratively solving the following system of equations:

$$\left(\frac{\partial D_u(\mathbf{q}^k(\mu))}{\partial \mathbf{q}} \right)^T \mathbf{J}(\mathbf{u}^k(\mu); \mu) \frac{\partial D_u(\mathbf{q}^k(\mu))}{\partial \mathbf{q}} \delta \mathbf{q}^k(\mu) = - \left(\frac{\partial D_u(\mathbf{q}^k(\mu))}{\partial \mathbf{q}} \right)^T \mathbf{R}(\mathbf{u}^k(\mu); \mu) \quad (14a)$$

$$\mathbf{q}^{k+1}(\mu) = \mathbf{q}^k(\mu) + \delta \mathbf{q}^k(\mu) \quad (14b)$$

$$\mathbf{q}^0(\mu) = E_u(\mathbf{u}^0(\mu)) \quad (14c)$$

2.4. Neural Network-Augmented Projection-Based ROM (PROM-ANN)

In this work, we focus on the PROM-ANN proposed by Barnett et al. [32]. This methodology constructs a nonlinear approximation manifold by augmenting a traditional linear reduced-order basis ROB with a nonlinear correction obtained via an artificial neural network (ANN). It builds upon earlier developments in quadratic approximation manifolds [29], addressing limitations inherent to purely linear ROM approximations, especially regarding problems exhibiting slowly decaying Kolmogorov n-width.

2.4.1. Construction of the Nonlinear Approximation Manifold

The PROM-ANN methodology constructs the solution manifold by leveraging a low-dimensional primary basis and enhancing it with a nonlinear mapping learned from data. Specifically, POD is employed by factorizing a snapshot matrix $\mathbf{S}_u \in \mathbb{R}^{N \times m}$ (containing m high-fidelity solutions) via a truncated singular value decomposition:

$$\mathbf{S}_u = \mathbf{U} \mathbf{\Sigma} \mathbf{V}^T, \quad (15)$$

where $\mathbf{U} \in \mathbb{R}^{N \times (n+\bar{n})}$ contains the left singular vectors, and $\mathbf{\Sigma}$ is the diagonal matrix of singular values σ_i , ordered by magnitude. From this decomposition, we construct the primary ROB $\mathbf{\Phi} \in \mathbb{R}^{N \times n}$ using the first n dominant modes, and a secondary ROB $\bar{\mathbf{\Phi}} \in \mathbb{R}^{N \times \bar{n}}$ using the next $\bar{n}$ subdominant modes. Given a full-order solution vector $\mathbf{u}^*(\mu) \in \mathbb{R}^N$, we approximate it with a nonlinear manifold representation defined as follows:

$$\mathbf{u}^*(\mu) \approx \mathbf{u}(\mu) = D_u(\mathbf{q}(\mu)) = \mathbf{u}_{\text{ref}} + \mathbf{\Phi} \mathbf{q}(\mu) + \bar{\mathbf{\Phi}} \mathcal{N}(\mathbf{q}(\mu); \Theta) \quad (16a)$$

$$\mathbf{q}(\mu) = E_u(\mathbf{u}^*(\mu)) = \mathbf{\Phi}^T (\mathbf{u}^*(\mu) - \mathbf{u}_{\text{ref}}), \quad (16b)$$

where $\mathbf{u}_{\text{ref}} \in \mathbb{R}^N$ is a suitable reference solution, typically chosen as the mean of the snapshots, $\mathbf{q}(\mu) \in \mathbb{R}^n$ are the primary reduced coordinates, encoding the dominant features of the solution, and $\mathcal{N}(\mathbf{q}; \Theta) : \mathbb{R}^n \rightarrow \mathbb{R}^{\bar{n}}$ is a neural network with parameters Θ , creating a nonlinear mapping from the primary to the secondary reduced coordinates, thus providing a nonlinear correction to the linear subspace approximation.

The ANN acts as a nonlinear map, capturing correlations between dominant (primary) and subdominant (secondary) mode coefficients. Rather than explicitly reconstructing truncated information, the ANN provides a nonlinear relationship that significantly improves the compactness and accuracy of the reduced-order representation.

To solve the reduced-order system defined in Equation (14), it is necessary to explicitly compute the derivative of the decoder function with respect to the primary reduced coordinates. For the PROM-ANN architecture defined in Equation (16), this derivative reads as follows:

$$\frac{\partial D_u(\mathbf{q}(\mu))}{\partial \mathbf{q}} = \mathbf{\Phi} + \bar{\mathbf{\Phi}} \frac{\partial \mathcal{N}(\mathbf{q}(\mu); \Theta)}{\partial \mathbf{q}}. \quad (17)$$

The derivative of the neural network term $\frac{\partial \mathcal{N}(\mathbf{q}(\mu); \Theta)}{\partial \mathbf{q}}$ can be computed efficiently using automatic differentiation frameworks typically employed in neural network training.

2.4.2. Training the Neural Network

The ANN is trained using pre-computed solution snapshots from the high-fidelity model. For each training snapshot indexed by $j = 1, \dots, m$, we first compute the primary and secondary reduced coordinates by projection onto their respective ROBs:

$$\mathbf{q}_j(\mu_j) = \Phi^T(\mathbf{u}_j^*(\mu_j) - \mathbf{u}_{\text{ref}}), \quad (18a)$$

$$\bar{\mathbf{q}}_j(\mu_j) = \bar{\Phi}^T(\mathbf{u}_j^*(\mu_j) - \mathbf{u}_{\text{ref}}). \quad (18b)$$

The original PROM-ANN approach [32] defines the training loss function directly on the discrepancy between the secondary coordinates $\bar{\mathbf{q}}_j$ and the ANN prediction $\mathcal{N}(\mathbf{q}_j; \Theta)$:

$$\mathcal{L}_{\text{PROM-ANN}} = \frac{1}{m} \sum_{j=1}^m \|\bar{\mathbf{q}}_j(\mu_j) - \mathcal{N}(\mathbf{q}_j(\mu_j); \Theta)\|^2. \quad (19)$$

While very computationally efficient, this approach lacks a proper normalization strategy in order to guarantee that the neural network can be effectively trained. We will study this issue in Section 4.

3. Discrete PINN-like Loss

Our nonlinear ROM architecture aims to incorporate the physics of the problem into the approximation manifold construction. The equivalent effect is accomplished in traditional PINNs by auto-differentiating the neural network's output with regard to a given point in continuous space and time, using the strong form of the PDE system. In contrast, a discrete approach like ours requires a numerical approximation through the discretization of the PDE, which can be performed via a variety of techniques such as the finite element or finite volume methods. The integration of these discretized residuals into neural networks places our approach within the category of informed machine learning, as detailed by [64].

The proposal to use discrete approximation for substituting the auto-differentiation in PINNs for residual minimization is introduced almost simultaneously in [44,45]. Both of these approaches introduce NN architectures for solving forward problems of linear, steady-state simulation and [44] additionally presents a model for backwards problems of the same nature. Their conceptualization does not differ too much from classical PINNs in terms of inputs, outputs, and loss definition: they take the parameters vector for the desired simulation as input, and return the results of the nodal variables of the system as output. The spatial information that is typically an input in PINNs is intrinsically defined in the FEM solver at the time of the residual computation.

In terms of the training loss, both approaches aim to minimize the mean squared L2-norm of the residual, as determined by the FEM solver. This minimization considers the predicted snapshot (nodal variables) and the relevant simulation parameters:

$$\mathcal{L}_{\mathbf{R}} = \frac{1}{mN} \sum_{j=1}^m \left\| \mathbf{R}(\mathcal{N}(\mu_j; \Theta); \mu_j) \right\|^2, \quad (20)$$

where Θ are the trainable parameters of the neural network conforming the PINN, index j specifies the different samples to be used within the batch, m is the number of samples in the batch, and N is the number of degrees of freedom in our system (and therefore the size of the residual). An important remark about this loss is that it requires the removal

of contributions in the residual $\mathbf{R}(\mathcal{N}(\boldsymbol{\mu}_j; \boldsymbol{\Theta}); \boldsymbol{\mu}_j)$ at the degrees of freedom with Dirichlet conditions; otherwise, the norm will not approach zero.

The exact implementation of this loss differs in both approaches. The one in [45] is limited to static linear cases of structural mechanics and proposes a specific loss scaling strategy for these cases. They also propose an implementation strategy to perform the loss computation in batches. Meanwhile, the approach in [44] is more general, but still only applicable to linear PDEs. This limitation to linear cases significantly simplifies the loss implementation, as the residual becomes of the form $\mathbf{R}(\mathbf{u}; \boldsymbol{\mu}) = \mathbf{K}(\boldsymbol{\mu})\mathbf{u} - \mathbf{F}(\boldsymbol{\mu})$. So, both the stiffness matrix $\mathbf{K}$ and the vector of external contributions $\mathbf{F}$ are independent of the current snapshot. Accordingly, the loss becomes the following:

$$\mathcal{L}_{\mathbf{R}} = \frac{1}{mN} \sum_{j=1}^m \left\| \mathbf{K}(\boldsymbol{\mu}_j) \mathcal{N}(\boldsymbol{\mu}_j; \boldsymbol{\Theta}) - \mathbf{F}(\boldsymbol{\mu}_j) \right\|^2 \quad (21)$$

where one could have pre-computed all $\mathbf{K}(\boldsymbol{\mu}_j)$ and $\mathbf{F}(\boldsymbol{\mu}_j)$, thus allowing the whole optimization to be performed via auto-differentiation, without calling the FEM software.

Even the approaches in more recent papers like [65,66] are still designed only for linear problems, with the difference being their specific use cases, the architectures they use for the surrogate model, and the loss being just the norm of the residual, instead of the square of it.

In contrast, our approach differs in two key aspects: (1) the loss function is formulated in a parameter-agnostic manner, and (2) it is designed to handle both linear and nonlinear cases, thereby enabling a much broader range of applications. As with PINNs, our method also allows for a combination of physics-based and data-driven losses. The following subsections discuss these three developments.

3.1. Parameter-Agnostic Loss

One aspect in which our methodology diverges significantly from the original idea of PINNs is that we are not training a self-contained surrogate model (typically in the form of a single neural network $\mathcal{N}(\boldsymbol{\mu}_j)$), but instead, the approximation manifold is defined by a trainable encoder–decoder pair $D_u(E_u(\mathbf{u}_j^*))$. As we can see, the latter case does not take the simulation parameters $\boldsymbol{\mu}_j$ as an input, so it is agnostic to them. This is because the FEM-based ROM software itself will be in charge of conditioning the simulation with those parameters and then finding the most appropriate solution within our latent space.

This is not the only difference in our methodology. Traditionally, for a loss like the one in Equation (20), the exact parameters $\boldsymbol{\mu}_j$ for the solution need to be specified, as these are what will yield a residual of zero and therefore enable the learning by minimization. Instead, we design a loss function in which the parameters applied in the FEM software can be arbitrary. In this new loss, we are not merely minimizing the properly parameterized residual of the predicted quantity $D_u(E_u(\mathbf{u}_j^*))$, but the difference between this quantity and the residual of the target solution $\mathbf{u}_j^*$, both with constant parameter $\boldsymbol{\mu}$:

$$\mathcal{L}_{\mathbf{R}} = \frac{1}{mN} \sum_{j=1}^m \left\| \mathbf{R}(D_u(E_u(\mathbf{u}_j^*)); \boldsymbol{\mu}) - \mathbf{R}(\mathbf{u}_j^*; \boldsymbol{\mu}) \right\|^2. \quad (22)$$

In this case, the trainable parameters $\boldsymbol{\Theta}$ are contained within the encoder and/or decoder of our ROM, and their specific form depends on the chosen architecture. Strictly speaking, we should write $D_u(E_u(\mathbf{u}_j^*; \boldsymbol{\Theta}); \boldsymbol{\Theta})$. However, for clarity and conciseness, we will omit $\boldsymbol{\Theta}$ from the notation in what follows.

Our approach aims to minimize the discrepancy in residual behavior when it is non-zero, ensuring that the ROM approximation manifold captures not only the converged solution but also the behavior of the residual outside of convergence situations. In our

approach, the decoder will be readily integrated into the Newton iterative procedure. Thus, we aim for it to enhance its residual representation near the convergence space, aiding in achieving an optimal converged solution. On the same line, we no longer have the restriction that we mentioned for Equation (20) on the components of the residual associated with the Dirichlet conditions. This means that we can leave those components in order to learn them too. The main caveat of using this approach is that it still requires the original HFM's snapshots samples $\mathbf{u}_j^*$, even when training via the residual. In our specific case, the encoder–decoder architecture of choice will need these snapshots regardless, so it is not a major inconvenient.

Regarding boundary conditions, it is important to consider that Dirichlet conditions must be imposed strongly in the solutions vector whenever we compute the residual. That means that whatever architecture we use as the encoder–decoder pair, it needs to operate on and predict only the degrees of freedom unaffected by Dirichlet conditions. Then, the fixed ones are given their corresponding value forcefully before computing the residual and Jacobian matrix.

3.2. In-Training Integration of FEM Software

The obstacle impeding the transition from linear to nonlinear cases in other FEM-based residual losses [44,45] is not a theoretical or mathematical difficulty from generating the residuals themselves or their derivatives. We have extensive FEM software at our disposal that already does precisely this. The difficulty is in dynamically integrating this information in an effective way during the training of the decoder.

Let us study the loss we proposed in Equation (22) in more detail to identify our needs during training. The loss itself needs the values for two residuals: $\mathbf{R}(D_u(E_u(\mathbf{u}_j^*)); \boldsymbol{\mu})$ and $\mathbf{R}(\mathbf{u}_j^*; \boldsymbol{\mu})$. The latter can be pre-computed, as it will be constant for sample j during the whole training. The first one, however, has to be computed within the FEM software with an updated snapshot value at each training step.

The actual training of the model happens during the backpropagation of the loss, which necessitates calculating the gradient of the loss with respect to $\boldsymbol{\Theta}$. This is typically carried out automatically thanks to the auto-differentiation capabilities of deep learning frameworks. However, as we take part of the loss computation onto external software, we are unable to do this and we have to code the gradient manually. The derivative of the loss function $\mathcal{L}_R$ can be decomposed using the chain rule:

$$\frac{\partial \mathcal{L}_R}{\partial \boldsymbol{\Theta}} = \frac{2}{mN} \sum_{j=1}^m \left(\mathbf{R}(\mathbf{u}_j; \boldsymbol{\mu}) - \mathbf{R}(\mathbf{u}_j^*; \boldsymbol{\mu}) \right)^T \frac{\partial \mathbf{R}(\mathbf{u}_j; \boldsymbol{\mu})}{\partial \mathbf{u}_j} \frac{\partial \mathbf{u}_j}{\partial \boldsymbol{\Theta}}. \quad (23)$$

where $\mathbf{u}_j = D_u(E_u(\mathbf{u}_j^*))$ is the current prediction for the nodal values of the solution.

The factor $\frac{\partial \mathbf{R}(\mathbf{u}_j; \boldsymbol{\mu})}{\partial \mathbf{u}_j}$ is the Jacobian matrix of the residual function with respect to the predicted solution vector $\mathbf{u}_j$. This Jacobian is computed and assembled entirely in the FEM software, given the current snapshot and arbitrary simulation parameter. Within the context of FEM, it is expressed as follows:

$$\mathbf{J}_F(\mathbf{u}_j; \boldsymbol{\mu}) = \frac{\partial \mathbf{R}(\mathbf{u}_j; \boldsymbol{\mu})}{\partial \mathbf{u}_j}. \quad (24)$$

The final factor $\frac{\partial \mathbf{u}_j}{\partial \boldsymbol{\Theta}}$ represents another Jacobian, this time of the predicted snapshot with respect to the trainable parameters. We can express it as follows:

$$\mathbf{J}_D(\mathbf{u}_j; \boldsymbol{\mu}) = \frac{\partial \mathbf{u}_j}{\partial \boldsymbol{\Theta}}. \quad (25)$$

This latter derivative is applied only on the computations performed in the trainable encoder–decoder pair $D_u(E_u(\mathbf{u}_j^*))$. As such, this factor is self-contained within the deep learning framework and does not require additional external data. Such a Jacobian could be obtained by using the `tf.GradientTape.batch_jacobian()` method in TensorFlow, or equivalent methodologies in other deep learning frameworks.

To compute the loss gradient as it is defined explicitly in Equation (23), we just need to collect the factors coming from the FEM software into the deep learning framework. However, this computation would be unnecessarily inefficient because of the two main reasons described next. For each of these, we propose a corresponding implementation strategy:

1. First, take into account that $\mathbf{J}_F(\mathbf{u}_j; \mu)$ is typically a high-dimensional and highly sparse matrix. So, moving it directly from one software to another could be costly, especially if it cannot be treated as a sparse structure in all computations. FEM software is typically designed to cope with these kinds of matrices and to perform sparse vector–matrix multiplications. Therefore, for each snapshot, we pre-compute the following quantities:

$$\begin{aligned}\mathbf{e}_{\mathbf{R},j} &:= \left(\mathbf{R}(\mathbf{u}_j; \mu) - \mathbf{R}(\mathbf{u}_j^*; \mu) \right)^T \\ \mathbf{v}_{\mathbf{R},j} &:= \mathbf{e}_{\mathbf{R},j} \mathbf{J}_F(\mathbf{u}_j; \mu)\end{aligned}\quad (26)$$

within the FEM software. Both of these are now vectorial quantities, making the transfer to the deep learning framework more efficient. $\mathbf{e}_{\mathbf{R},j}$ is the error used to compute the loss, as $\mathcal{L}_{\mathbf{R}} = \frac{1}{mN} \sum_{j=1}^m \|\mathbf{e}_{\mathbf{R},j}\|^2$. And $\mathbf{v}_{\mathbf{R},j}$ will be used to compute the gradient of the loss.

2. The main bottleneck comes from needing to compute m full Jacobian matrices $\{\mathbf{J}_D(\mathbf{u}_j; \mu)\}_{j=1}^m$ via auto-differentiation, instead of just the gradient of a single scalar $\mathcal{L}_{\mathbf{R}}$. Avoiding this is straightforward when reformulating the gradient of the loss with vectors $\mathbf{v}_{\mathbf{R},j}$. We realize that, to the deep learning framework, externally computed $\mathbf{v}_{\mathbf{R},j}$ is no longer seen as a function depending on the model parameters Θ , but as a constant instead. This means that we can treat it as such during the computation and obtain the following:

$$\frac{\partial \mathcal{L}_{\mathbf{R}}}{\partial \Theta} = \frac{2}{mN} \sum_{j=1}^m \mathbf{v}_{\mathbf{R},j} \frac{\partial \mathbf{u}_j}{\partial \Theta} = \frac{\partial}{\partial \Theta} \left(\frac{2}{mN} \sum_{j=1}^m \mathbf{v}_{\mathbf{R},j} \mathbf{u}_j \right) \quad (27)$$

Therefore, we compute a single gradient of a scalar value for the whole batch via auto-differentiation.

In our case in particular, we take KratosMultiphysics as our choice of FEM software. As it is open-source, it enables us to develop all the custom functionalities needed to take a given set of snapshots $\{\mathbf{u}_j\}_{j=1}^m$ (those of the current batch), and for each one apply it as the current solution to then output $\mathbf{e}_{\mathbf{R},j}$ and $\mathbf{v}_{\mathbf{R},j}$.

3.3. Data-Based Loss Term

Even when training with the residual loss, it might be beneficial to introduce a data-related loss simultaneously. This is common in traditional PINNs in two instances: It is used specifically on the boundary and initial conditions to try to enforce Dirichlet conditions [37] (unless a special approach is used to establish them strongly [63]), and it is also used on collocation points sampled inside the simulation space as a regularization that can help the overall loss converge faster [38].

In our case, the Dirichlet conditions are enforced strongly and managed by the FEM software for their effect on the residual. However, we could still introduce such a loss term for regularization purposes. Thus, the total loss function is defined by two components:

$$\mathcal{L} = \omega_{\mathbf{R}} \mathcal{L}_{\mathbf{R}} + \omega_d \mathcal{L}_d. \quad (28)$$

where $\omega_{\mathbf{R}}$ and ω_d are tunable hyper-parameters to balance both terms as needed, and the data-related loss $\mathcal{L}_d$ is defined simply as the mean squared error between the currently predicted snapshot $\mathbf{u}_j = D_u(E_u(\mathbf{u}_j^*))$ and the ground truth one $\mathbf{u}_j^*$:

$$\mathcal{L}_d = \frac{1}{mN} \sum_{j=1}^m \|\mathbf{u}_j - \mathbf{u}_j^*\|^2. \quad (29)$$

Then, the implementation of the total loss $\mathcal{L}$ and its gradient starts by defining the following vectors as constants:

$$\begin{aligned} \mathbf{e}_{d,j} &= \mathbf{v}_{d,j} := (\mathbf{u}_j - \mathbf{u}_j^*)^T, \\ \mathbf{e}_{\mathbf{R},j} &:= (\mathbf{R}(\mathbf{u}_j; \boldsymbol{\mu}) - \mathbf{R}(\mathbf{u}_j^*; \boldsymbol{\mu}))^T, \\ \mathbf{v}_{\mathbf{R},j} &:= \mathbf{e}_{\mathbf{R},j} \mathbf{J}_F(\mathbf{u}_j; \boldsymbol{\mu}), \end{aligned} \quad (30)$$

and then using these to compute the loss as

$$\mathcal{L} = \frac{1}{mN} \sum_{j=1}^m \left(\omega_{\mathbf{R}} \|\mathbf{e}_{\mathbf{R},j}\|^2 + \omega_d \|\mathbf{e}_{d,j}\|^2 \right) \quad (31)$$

Finally, its gradient via auto-differentiation is as follows:

$$\frac{\partial \mathcal{L}}{\partial \boldsymbol{\Theta}} = \frac{\partial}{\partial \boldsymbol{\Theta}} \left(\frac{2}{mN} \sum_{j=1}^m \left(\omega_{\mathbf{R}} \mathbf{v}_{\mathbf{R},j} + \omega_d \mathbf{v}_{d,j} \right) \mathbf{u}_j \right). \quad (32)$$

4. Modifications to the PROM-ANN Architecture

We adopt the PROM-ANN architecture from [32] as the foundation for developing our physics-informed ROM model, which introduces the use of neural networks while ensuring scalability and avoiding the requirement of a structured mesh for the underlying simulation. The architecture was introduced in Section 2.4, where we noted that it can be interpreted as a nonlinear extension of classical POD that introduces additional effective modes without increasing the size of the latent space.

However, we find that the architecture, as originally proposed, is difficult to train—at least in our case, where the underlying problem exhibits a rapidly decaying singular value spectrum. To address this, we first revise both the architecture and the data-driven loss formulation to improve training capability. Later, in the following section, we will incorporate our residual-based loss into the framework.

The modifications we propose in this section are as follows: (1) scaling of reduced coefficients, (2) changing the data-based loss for one that takes the full snapshot into account, and (3) properly scaling the loss.

4.1. Scaling of Reduced Coefficients

The main concern for us from the original architecture is that there is no normalization or scaling of the reduced coefficients prior to using them in the neural network. Ideally, neural networks are designed to operate on inputs that are approximately independent and identically distributed (i.i.d.), as this assumption facilitates more effective learning

and optimization [67,68]. This can rarely be guaranteed, but it is still common practice to perform some sort of normalization or scaling on the inputs so that they all operate in a similar range of values. Otherwise, serious issues may arise during training, mainly because of some of the inputs are ignored.

In the case of PROM-ANN, the inputs and outputs of the neural network are the coefficients for a given snapshot (remember from Section 2.4 that we train a network such that $\mathcal{N}(\Phi^T \mathbf{u}_j^*) \approx \tilde{\Phi}^T \mathbf{u}_j^*$). These coefficients, $\mathbf{q}_j^* = \Phi^T \mathbf{u}_j^*$ and $\tilde{\mathbf{q}}_j^* = \tilde{\Phi}^T \mathbf{u}_j^*$, will scale similarly to the singular value decay observed when performing SVD on the training data. Essentially, for a fast-decaying singular value profile, each coefficient will result in a considerably smaller range of coefficients than the previous one, causing the neural network to learn only from a few of the first ones.

In order to correct this issue, we modify the architecture of the encoder and the decoder themselves to include a pair of scaling matrices:

$$\begin{aligned} E_u(\mathbf{u}^*) &= \mathbf{q} = \Xi^{-1} \Phi^T \mathbf{u}^* \\ D_u(\mathbf{q}) &= \Phi \Xi \mathbf{q} + \tilde{\Phi} \tilde{\Xi} \mathcal{N}(\mathbf{q}) \end{aligned} \quad (33)$$

where both the projection matrices $\Phi, \tilde{\Phi}$ and the scaling matrices $\Xi, \tilde{\Xi}$ come from the SVD decomposition of the snapshots matrix:

$$\begin{aligned} SVD(\mathbf{S}_u) &= \mathbf{U} \Sigma \mathbf{V}^T \\ \Phi &= [\mathbf{U}_1 | \dots | \mathbf{U}_n], \quad \tilde{\Phi} = [\mathbf{U}_{n+1} | \dots | \mathbf{U}_{n+\tilde{n}}] \\ \Xi &= \frac{1}{\sqrt{M}} \begin{bmatrix} \sigma_1 & & & \\ & \sigma_2 & & \\ & & \dots & \\ & & & \sigma_n \end{bmatrix}, \quad \tilde{\Xi} = \frac{1}{\sqrt{M}} \begin{bmatrix} \sigma_{n+1} & & & \\ & \sigma_{n+2} & & \\ & & \dots & \\ & & & \sigma_{n+\tilde{n}} \end{bmatrix} \end{aligned} \quad (34)$$

and σ_i represents the singular values found in the diagonal of the Σ matrix. Finally, M is the total number of samples in $\mathbf{S}_u$.

This procedure effectively rescales each coefficient to have a roughly equivalent range. In the ideal case where all rows of the training snapshot matrix $\mathbf{S}_u$ have zero mean, multiplying by the matrix $\tilde{\Xi}^{-1}$ scales the quantity $\mathbf{Q} = \tilde{\Xi}^{-1} \mathbf{U}^T \mathbf{S}_u$ such that the covariance matrix between its rows becomes the identity. In other words, the modes in $\mathbf{Q}$ are uncorrelated—already ensured by the projection onto $\tilde{\Phi}$ —and have unit variance.

In our case, the condition that all row means in $\mathbf{S}_u$ are exactly zero does not hold. Nevertheless, as we will show in Section 6, the resulting scaling still leads to reduced coefficients with similar magnitudes in our particular use case.

We intentionally avoid enforcing zero-mean values, as we do not wish to apply any offset to the projected values. This design choice is crucial because it enables a neural network architecture without biases, thereby ensuring that $D_u(E_u(\mathbf{0})) = \mathbf{0}$ holds. Additionally, the proposed scaling procedure preserves the orthogonality of the projection matrices, so we still have $E_u(D_u(\mathbf{q})) = \mathbf{q}$.

Finally, no additional computation is required to obtain the scaling factors, since the SVD is already performed as part of the original methodology. In this sense, the approach is highly efficient.

Note that this formulation makes the assumption that the problem is homogenous, which is true for our particular use case. Still, it can be applied to non-homogenous cases. In such cases, the user should remove the components affected by Dirichlet conditions from all snapshots and proceed as stated. Then, at the time of running the online simulation, the fixed components of the solution can be imposed strongly, as they are known.

4.2. Corrected Data-Based Loss

Now we have an architecture that provides appropriately scaled features for the neural network. But at the same time, this scaling may not be the best if we want to apply the training loss as in the original paper [32]; that is, measuring the error on the predicted $\hat{\mathbf{q}}_j$ coefficients themselves. This loss would translate like this to our architecture:

$$\mathcal{L}'_d = \frac{1}{mN} \sum_{j=1}^m \left\| \mathcal{N}(\mathbf{q}_j^*) - \hat{\Xi}^{-1} \bar{\Phi}^T \mathbf{u}_j^* \right\|^2 = \frac{1}{mN} \sum_{j=1}^m \left\| \mathcal{N}(\Xi^{-1} \Phi^T \mathbf{u}_j^*) - \hat{\Xi}^{-1} \bar{\Phi}^T \mathbf{u}_j^* \right\|^2 \quad (35)$$

The rationale behind dividing by N is to ensure consistency with the loss formulations introduced in Section 3.

Such a loss will give the same importance to all of the output features, or reduced coefficients in this case. We know from the nature of POD that this is not desirable, as lower modes should be given more importance than higher ones. In fact, we now must reverse, within the loss, the scaling that we applied to the reduced coefficients. We can do so by applying $\hat{\Xi}$ on the contents of the norm in Equation (35):

$$\mathcal{L}_{d,\text{compact}} = \frac{1}{mN} \sum_{j=1}^m \left\| \hat{\Xi} \left(\mathcal{N}(\mathbf{q}^*) - \hat{\Xi}^{-1} \bar{\Phi}^T \mathbf{u}^* \right) \right\|^2 \quad (36)$$

But we could also achieve the same effect by just enforcing the whole reconstructed full-order snapshot to approximate the ground truth one; that is, using exactly the same data-based loss that we had defined in Section 3.3:

$$\mathcal{L}_d = \frac{1}{mN} \sum_{j=1}^m \left\| \mathbf{u}_j - \mathbf{u}_j^* \right\|^2 = \frac{1}{mN} \sum_{j=1}^m \left\| \Phi \Xi \mathbf{q}^* + \bar{\Phi} \hat{\Xi} \mathcal{N}(\mathbf{q}^*) - \mathbf{u}_j^* \right\|^2 \quad (37)$$

This second loss is more intuitive, as we make it explicit to achieve the final goal for our ROM approximation manifold: learning the full snapshot itself. However, it is not difficult to prove that both Equations (36) and (37) are equivalent up to a constant offset; that is, assuming that all snapshots $\{\mathbf{u}_j^*\}_{j=1}^m$ were included in the set $\mathbf{S}_u$ to which we performed the SVD.

Proof. By developing from $\mathcal{L}_d$, we obtain the following:

$$\mathcal{L}_d = \frac{1}{mN} \sum_{j=1}^m \left\| \mathbf{u}_j - \mathbf{u}_j^* \right\|^2 = \frac{1}{mN} \sum_{j=1}^m \left\| \Phi \Xi \mathbf{q}^* + \bar{\Phi} \hat{\Xi} \mathcal{N}(\mathbf{q}^*) - \mathbf{u}_j^* \right\|^2$$

By developing $\mathbf{u}_j^* = (\Phi \Xi \Xi^{-1} \Phi^T + \bar{\Phi} \hat{\Xi} \hat{\Xi}^{-1} \bar{\Phi}^T + \hat{\Phi} \hat{\Xi} \hat{\Xi}^{-1} \hat{\Phi}^T) \mathbf{u}_j^*$. Where $\hat{\Phi}$ is the orthonormal basis containing the singular vectors from $SVD(\mathbf{S}_u)$ that were not included in neither Φ or $\bar{\Phi}$, and $\hat{\Xi}$ is analog but for the singular values matrix. Then, applying this obtains the following:

$$\begin{aligned} \mathcal{L}_d &= \frac{1}{mN} \sum_{j=1}^m \left\| \Phi \Xi \mathbf{q}^* + \bar{\Phi} \hat{\Xi} \mathcal{N}(\mathbf{q}^*) - \left(\Phi \Xi \mathbf{q}^* + \bar{\Phi} \hat{\Xi} \hat{\Xi}^{-1} \bar{\Phi}^T \mathbf{u}_j^* + \hat{\Phi} \hat{\Xi} \hat{\Xi}^{-1} \hat{\Phi}^T \mathbf{u}_j^* \right) \right\|^2 \\ &= \frac{1}{mN} \sum_{j=1}^m \left\| \bar{\Phi} \hat{\Xi} \left(\mathcal{N}(\mathbf{q}^*) - \hat{\Xi}^{-1} \bar{\Phi}^T \mathbf{u}_j^* \right) - \hat{\Phi} \hat{\Phi}^T \mathbf{u}_j^* \right\|^2 \end{aligned}$$

By acknowledging that $\bar{\Phi}$ and $\hat{\Phi}$ are orthogonal to each other, we obtain the following:

$$\mathcal{L}_d = \frac{1}{mN} \sum_{j=1}^m \left(\left\| \bar{\Phi} \hat{\Xi} \left(\mathcal{N}(\mathbf{q}^*) - \hat{\Xi}^{-1} \bar{\Phi}^T \mathbf{u}_j^* \right) \right\|^2 + \left\| \hat{\Phi} \hat{\Phi}^T \mathbf{u}_j^* \right\|^2 \right)$$

By acknowledging that the L^2 norm is invariant to the right-multiplication of an orthonormal matrix and that the terms $\|\hat{\Phi}\hat{\Phi}^T \mathbf{u}_j^*\|$ do not depend on trainable parameters Θ , we obtain the following:

$$\mathcal{L}_d = \frac{1}{mN} \sum_{j=1}^m \left\| \hat{\Xi} \left(\mathcal{N}(\mathbf{q}^*) - \hat{\Xi}^{-1} \hat{\Phi}^T \mathbf{u}_j^* \right) \right\|^2 + C = \mathcal{L}_{d,\text{compact}} + C$$

□

If the user does not intend on applying physics-informed training, then choosing $\mathcal{L}_{d,\text{compact}}$ would make the most sense because of its reduced complexity. However, the fact that $\mathcal{L}_d$ takes the snapshots to the full space within the computation makes it most appropriate to be paired with a physics-based loss, which will require the full vector of nodal solutions anyway. In any of the two cases, the ideal method would be to pre-compute $\mathbf{q}_j^*$ for all samples to improve efficiency.

4.3. Scaling of the Data-Based Loss

As written in Equation (37), $\mathcal{L}_d$ would exhibit high variability depending on the number of modes included in the primary ROB. This is because, by including more modes in Φ while following their importance order, we are heavily limiting the possible error $\|\mathbf{u}_j - \mathbf{u}_j^*\|^2$ that we can have (this is true for snapshots within the set $\mathbf{S}_u$, and should reflect in new samples if they are properly represented by the POD).

In order to correct this issue, we propose to use a pre-computed scaling factor that will be applied globally to the loss:

$$\mathcal{L}_d = \frac{1}{mN} \frac{1}{e_{\text{POD},d}} \sum_{j=1}^m \|\mathbf{u}_j - \mathbf{u}_j^*\|^2. \quad (38)$$

With the new factor $e_{\text{POD},d}$ defined as follows:

$$e_{\text{POD},d} := \frac{1}{MN} \sum_{j=1}^M \|\Phi \Phi^T \mathbf{u}_j^* - \mathbf{u}_j^*\|^2. \quad (39)$$

This quantity corresponds to the mean squared reconstruction error of the standard POD approach when only the primary modes are retained. This is averaged over all the samples in the training set, not only those in the batch, in order to make it representative for any choice of samples during the training, and thus means that it only has to be computed once at the beginning.

This choice of scaling acts as a safeguard preventing the gradients from becoming too low, which could induce numerical instabilities during training, and prevents the weight updates from varying wildly in scale based on the chosen numbers of primary modes for the same learning rate. Apart from this, it also gives a much more interpretable value to the loss, as it will become an indicator to how much better the current model is compared to just using POD for the same size of latent space. In this sense, it is helpful when troubleshooting, as a value over 1 would clearly indicate that we are losing accuracy compared to POD.

4.4. Online Phase of Nonlinear ROM

In order to perform the online ROM simulation, we need to adapt the nonlinear iteration problem to the new architecture. Essentially, we substitute our decoder into the Galerkin-based nonlinear iteration formulation for nonlinear manifolds (as seen in Equation (14)). The result would be as follows:

$$\left(\Phi \Xi + \bar{\Phi} \bar{\Xi} \frac{\partial \mathcal{N}(\mathbf{q}^k)}{\partial \mathbf{q}} \right)^T \mathbf{J}(\mathbf{u}(\mu); \mu) \left(\Phi \Xi + \bar{\Phi} \bar{\Xi} \frac{\partial \mathcal{N}(\mathbf{q}^k)}{\partial \mathbf{q}} \right) \delta \mathbf{q}^k(\mu) = - \left(\Phi \Xi + \bar{\Phi} \bar{\Xi} \frac{\partial \mathcal{N}(\mathbf{q}^k)}{\partial \mathbf{q}} \right)^T \mathbf{R}(\mathbf{u}(\mu); \mu) \quad (40)$$

$$\mathbf{q}^{k+1}(\mu) = \mathbf{q}^k(\mu) + \delta \mathbf{q}^k(\mu)$$

meaning that at each nonlinear iteration, we have to recompute $\bar{\Phi} \bar{\Xi} \frac{\partial \mathcal{N}(\mathbf{q}^k)}{\partial \mathbf{q}}$.

The online ROM simulation is performed entirely within the FEM software, Kratos-Multiphysics, as going back between this and the deep learning software would be unnecessarily expensive. Therefore, we made custom methods within KratosMultiphysics to compute both the forward pass of the neural network, $\mathcal{N}(\mathbf{q})$, and the Jacobian of its outputs with respect to the inputs, $\frac{\partial \mathcal{N}(\mathbf{q})}{\partial \mathbf{q}}$. The other quantities are defined at initialization from the training results and are kept constant.

5. Integration of Physics-Based Loss into PROM-ANN

Here, we take the developments carried out in the two last sections and join them in order to finally obtain a method to perform physics-informed nonlinear projection-based ROM. Essentially, we will take the total $\mathcal{L}$ loss developed in Section 3 and apply it on the PROM-ANN formulation developed in Section 4 while keeping in mind the adaptations to the loss that we also performed in this latter section.

The full discrete PINN-like loss $\mathcal{L}'$, as formulated in Section 3, is as follows:

$$\mathcal{L}' = \omega_{\mathbf{R}} \mathcal{L}'_{\mathbf{R}} + \omega_d \mathcal{L}'_d = \frac{\omega_{\mathbf{R}}}{mN} \sum_{j=1}^m \left\| \mathbf{R}(\mathbf{u}_j; \mu) - \mathbf{R}(\mathbf{u}_j^*; \mu) \right\|^2 + \frac{\omega_d}{mN} \sum_{j=1}^m \left\| \mathbf{u}_j - \mathbf{u}_j^* \right\|^2. \quad (41)$$

In Section 4.3, we determined that a full-snapshot data-based loss like $\mathcal{L}'_d$ in Equation (41) is appropriate to use once scaled by factor $1/e_{\text{POD},d}$. Therefore, this is the only modification we have to make on $\mathcal{L}'_d$.

But now, in order to still be able to use the residual-based loss term $\mathcal{L}'_{\mathbf{R}}$ appropriately, we should also scale it in a similar way to $\mathcal{L}'_d$. We propose modifying it as follows:

$$\mathcal{L}_{\mathbf{R}} = \frac{1}{mN} \frac{1}{e_{\text{POD},\mathbf{R}}} \sum_{j=1}^m \left\| \mathbf{R}(\mathbf{u}_j; \mu) - \mathbf{R}(\mathbf{u}_j^*; \mu) \right\|^2. \quad (42)$$

With $e_{\text{POD},\mathbf{R}}$ defined as the mean square error compared to the traditional POD but this time in terms of the residual and not the snapshot itself:

$$e_{\text{POD},\mathbf{R}} = \frac{1}{MN} \sum_{j=1}^M \left\| \mathbf{R}(\Phi \Phi^T \mathbf{u}_j^*; \mu) - \mathbf{R}(\mathbf{u}_j^*; \mu) \right\|^2. \quad (43)$$

By making the losses relative via $e_{\text{POD},\mathbf{R}}$ and $e_{\text{POD},d}$, the scale for both $\mathcal{L}_{\mathbf{R}}$ and $\mathcal{L}_d$ should be of a similar scale. With this small modification, the full loss becomes as follows:

$$\mathcal{L} = \omega_{\mathbf{R}} \mathcal{L}_{\mathbf{R}} + \omega_d \mathcal{L}_d = \frac{\omega_{\mathbf{R}}}{mN} \frac{1}{e_{\text{POD},\mathbf{R}}} \sum_{j=1}^m \left\| \mathbf{R}(\mathbf{u}_j; \mu) - \mathbf{R}(\mathbf{u}_j^*; \mu) \right\|^2 + \frac{\omega_d}{mN} \frac{1}{e_{\text{POD},d}} \sum_{j=1}^m \left\| \mathbf{u}_j - \mathbf{u}_j^* \right\|^2. \quad (44)$$

And the practical implementation of the loss itself and its gradient is as follows:

$$\mathcal{L} = \frac{1}{mN} \sum_{j=1}^m \left(\frac{\omega_{\mathbf{R}}}{e_{\text{POD},\mathbf{R}}} \left\| \mathbf{e}_{\mathbf{R},j} \right\|^2 + \frac{\omega_d}{e_{\text{POD},d}} \left\| \mathbf{e}_{d,j} \right\|^2 \right),$$

$$\frac{\partial \mathcal{L}}{\partial \Theta} = \frac{\partial}{\partial \Theta} \left(\frac{2}{mN} \sum_{j=1}^m \left(\frac{\omega_{\mathbf{R}}}{e_{\text{POD},\mathbf{R}}} \mathbf{v}_{\mathbf{R},j} + \frac{\omega_d}{e_{\text{POD},d}} \mathbf{v}_{d,j} \right) \mathbf{u}_j \right), \quad (45)$$

where all vector quantities $\mathbf{e}_{R,j}$, $\mathbf{e}_{d,j}$, $\mathbf{v}_{R,j}$, and $\mathbf{v}_{d,j}$ are exactly as defined in Equation (30) and treated as constants. And the definitions of the encoder and decoder are taken from Equation (33).

As we only modified the loss for the offline stage of ROM, the online stage stays untouched. Therefore, once the decoder is trained, we would proceed exactly as in Section 4.4 with new cases to simulate.

6. Use Case and Evaluation Methodology

The evaluation of our method is performed on a quasi-static structural mechanics case, specifically a nonlinear case of hyperelasticity. It simulates the deformation in a 2D rubber cantilever that is fixed at its left wall which has two different and perpendicularly oriented line loads, P_x and P_y , applied to its right end. The cantilever is defined by an unstructured mesh with 797 nodes, and the nodal variables to compute are the displacements in components X and Y. Therefore, our FOM system is of dimension $\mathcal{N} = 1594$. Figure 1 shows a schematic of the described setup.

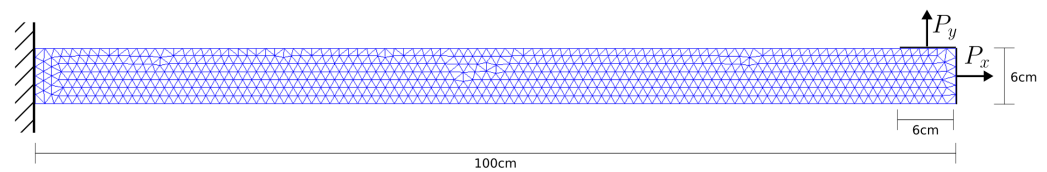


Figure 1. Mesh of the cantilever with line loads.

We define the parametric space as the range of loads that can be applied in each direction: $\mathcal{P} = [-3000, 3000] \times [-3000, 3000]$ N/m. And the displacements are computed in reference to the unperturbed position of each node $\mathbf{u}(P_x = 0, P_y = 0)$. Figure 2 shows the deformation on the cantilever for a random set of line load combinations within the parametric space.

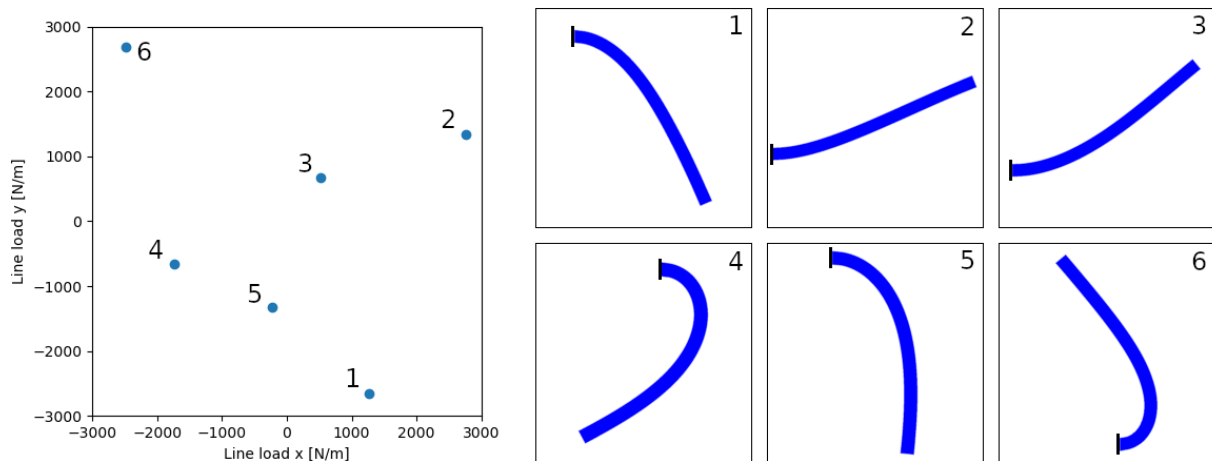


Figure 2. Examples of deformation in the cantilever for six different random combinations of line loads.

In order to build the snapshot matrix for the training, we perform the ROM simulations of 5000 different cases with parameters defined by a 2D Halton pseudo-random number generator. From each of these simulations, we store both the full snapshot $\mathbf{u}_j^* \in \mathbb{R}^N$ and the corresponding residual $\mathbf{R}(\mathbf{u}_j^*; \boldsymbol{\mu}) \in \mathbb{R}^N$. For the validation set, we generate 1250 different samples using the same strategy, and we generate 300 more for the test set. We perform all

evaluations over the test dataset; the training and the validation ones are used exclusively during the training process.

A reduced dataset with one order of magnitude fewer FOM simulations (i.e., 500 samples) was also evaluated (see Appendix A). While this resulted in a moderate decrease in accuracy, the PROM-ANN approach remained consistently more accurate than POD. Reducing the dataset by yet another order of magnitude introduces significant challenges—particularly for training the neural network—since learning a reliable nonlinear mapping (e.g., from 10 to 40 latent coordinates) from only 50 samples is a highly underdetermined task. The choice of 5000 samples was motivated by the low cost of the benchmark and the data demands of neural network training [68]. Adaptive sampling techniques (e.g., [69]) could be explored in future work to reduce offline cost.

Φ , $\bar{\Phi}$, Ξ , and $\bar{\Xi}$ are obtained from the SVD of the snapshots dataset for training. Figure 3 shows the decay of the singular values' energy for the first 200 modes of the SVD. The number of modes in each projection matrix is chosen by taking into account the accuracy obtained via traditional POD-based ROM using the same amount of modes. We establish two limit cases: one in which we select a value of $n = 6$, which would lead to a $\sim 0.1\%$ error on the displacements snapshot using POD, and one with $n = 20$, which would achieve a relative error of $\sim 10^{-6}$. We then take a series of n values in between these to compare in our experiments. The secondary ROM basis contains a variable amount of modes so that $n + \bar{n} = 60$ always.

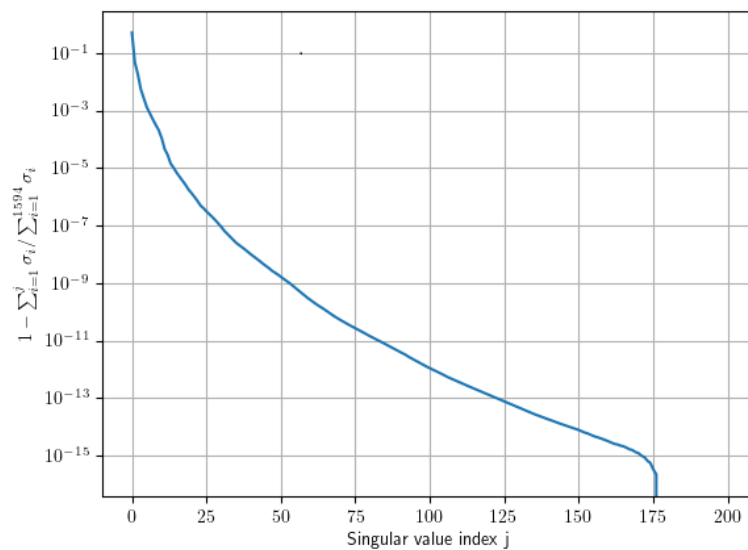


Figure 3. Singular values' energies for the first 200 modes of S_u .

Two different error metrics are defined:

- Relative error on snapshot:

$$e_u = \exp \left(\frac{1}{M} \sum_{j=1}^M \ln \frac{\|\mathbf{u}_j - \mathbf{u}_j^*\|}{\|\mathbf{u}_j^*\|} \right) \quad (46)$$

This is the geometric mean of the relative errors of each ROM sample $\mathbf{u}_j$ compared to the FOM one $\mathbf{u}_j^*$.

- Relative error on residual:

$$e_R = \exp \left(\frac{1}{M} \sum_{j=1}^M \ln \frac{\|\mathbf{R}(\mathbf{u}_j; \boldsymbol{\mu}) - \mathbf{R}(\mathbf{u}_j^*; \boldsymbol{\mu})\|}{\|\mathbf{R}(\mathbf{u}_j^*; \boldsymbol{\mu})\|} \right) \quad (47)$$

Again, this is the geometric mean of the samples' relative errors, but this time comparing the residual.

We will use these metrics in the next section in order to compare the behavior of our models both in reconstruction and online ROM.

In terms of software, we use KratosMultiphysics [58] as our FEM framework as it is open-source and lets us implement the required methods and interfaces in Python. For the neural network framework, we choose TensorFlow.

All neural networks presented contain two hidden layers of 200 neurons each with no bias. The architecture was selected based on empirical tuning through several trials using the snapshot-based loss. For the residual-based loss, the only hyper-parameter modification was a reduction in the initial learning rate to prevent overwriting previously learned weights, as this stage is intended as a fine-tuning pass. We use the Exponential Linear Unit (ELU) as the activation function following its successful application in the related literature [28,32]. ELU is continuously differentiable, being twice differentiable almost everywhere, and avoids vanishing gradients by allowing small negative outputs, addressing the known limitations of ReLU [70]. While no explicit comparison with other activations was conducted, we believe that functions with similar smoothness and gradient-preserving properties—such as Swish [71]—would behave similarly in this context. We implemented ELU directly within our FEM software to allow for ANN-PROM online simulation without the need for external software. A sinusoidal learning rate scheduling strategy is applied, reducing the learning rate down to 10^{-6} , and the AdamW optimizer is used with TensorFlow's default parameters. All trainings are performed with batches of size $m = 16$.

We emphasize that the proposed methodology is designed and validated exclusively for forward parametric simulations.

7. Results

In this section, we will go over the evaluation of the methods we have proposed throughout the paper. The first subsection will study the modifications to the original PROM-ANN [32] that were explained in Section 4. After that, we study the effect of training the architecture with the novel residual loss, as developed within Section 5. Finally, we carry out a performance assessment.

7.1. Modifications on the Original PROM-ANN

In this section, we examine the modifications introduced to the original PROM-ANN architecture during Section 4. The modification first involved the incorporation of scaling matrices Ξ and $\tilde{\Xi}$ within the encoder–decoder pair and within the loss. Then, we added a global, scalar scaling $1/e_{\text{POD},d}$ to the data-based loss. The main effects of these should be (1) achieving a similar range of values for all the neural network's inputs and (2) to make the loss and its gradients invariant to the choice of size for the latent space.

In order to confirm effect (1), we apply the encoder E_u on all snapshots from our training set, with fixed latent size $n = 60$. We perform this with the encoder as defined in the original POD-ANN paper [32] (specified in Equation (16)) and also with ours (specified in Equation (33)). Thus, obtaining $\mathbf{q}_{\text{orig},j} = \Phi^T \mathbf{u}_j^* \in \mathbb{R}^{60}$ and $\mathbf{q}_j = \Xi^{-1} \Phi^T \mathbf{u}_j^* \in \mathbb{R}^{60}$, respectively, for all the snapshots. Then, in Figure 4, we plot the statistics of the obtained values

for each component or mode in $\mathbf{q}_{\text{orig},j}$ and $\mathbf{q}_j$, side by side. For the original implementation (Figure 4a), only the first few modes have similarly large coefficients; they then become too small to differentiate. In contrast, by incorporating the singular values matrix, we obtain the result in Figure 4b, where all ranges are similarly large. These are, in fact, the range of values that would go into the neural network in each architecture.

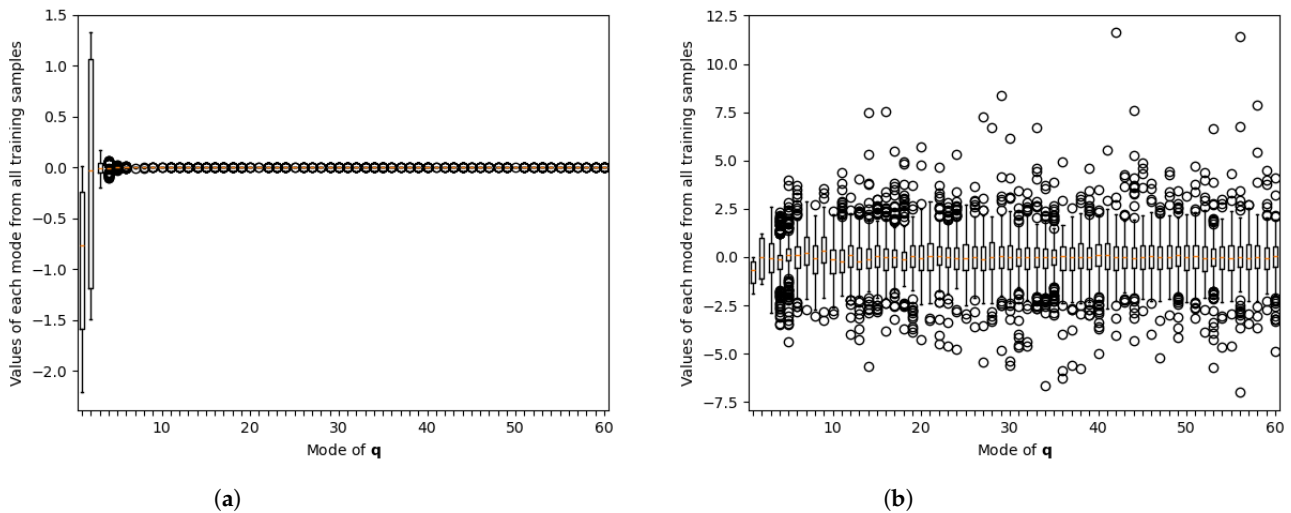


Figure 4. Boxplots showing the spread of the values for each mode of $\mathbf{q}$ for all the samples in our training dataset. (a) shows the case where $\mathbf{q} = \Phi^T \mathbf{u}^*$ and (b) the case where $\mathbf{q} = \Xi^{-1} \Phi^T \mathbf{u}^*$. The ranges are much more uniform in the latter case, which should be beneficial when training the neural network.

Moving on to effect (2), we perform a similar study but in the range of the mean of the gradients $\frac{\partial \mathcal{N}(\mathbf{q}_j; \Theta)}{\partial \Theta}$ obtained during backpropagation in the training routine. We compute the mean gradient for each batch, and then take the mean for all batches in the training epoch. Thus, collecting a single mean gradient value per epoch. We compare the trainings with and without scaling factors $1/e_{\text{POD},d}$ and $1/e_{\text{POD},R}$ (otherwise as $\mathcal{L}$ in Equation (44)). And for each of these two cases, we run four different training routines:

- S6: 6 primary modes ($n = 6$). Trained only on the data-based loss ($\omega_d = 1, \omega_R = 0$).
- S20: 20 primary modes ($n = 20$). Trained only on the data-based loss ($\omega_d = 1, \omega_R = 0$).
- R6: 6 primary modes ($n = 6$). Trained only on the residual loss ($\omega_d = 0, \omega_R = 1$).
- R20: 20 primary modes ($n = 20$). Trained only on the residual loss ($\omega_d = 0, \omega_R = 1$).

All of these are trained for 800 epochs. We can see in Figure 5 the comparison of the statistics of these gradients for the different training modalities without the scalings (Figure 5a) and with the scalings (Figure 5b). While the former presents orders of magnitude in difference for these values depending on the size of the latent space, the latter one provides very uniform results.

The final step in the evaluation is to examine how these modifications altogether affect the resulting decoders after training. That is, the capacity for the trained decoders to reconstruct the original snapshots, and also to serve as the approximation manifolds within ROM simulations. For this, we compare three architectures and training losses:

- *s-loss*: Using the encoder–decoder pair introduced by us in Equation (33), train via the data-based loss $\mathcal{L}_d$ as defined in (38) for 800 epochs, with the learning rate starting at 10^{-3} .
- *q-loss*: Using the original PROM-ANN encoder–decoder pair, as in Equation (16), train via the original loss $\mathcal{L}_{\text{PROM-ANN}}$ as defined in Equation (19) for 800 epochs, with the learning rate starting at 10^{-3} .

- *POD*: Using the fully linear POD encoder–decoder pair. This involves no training apart from the SVD computation.

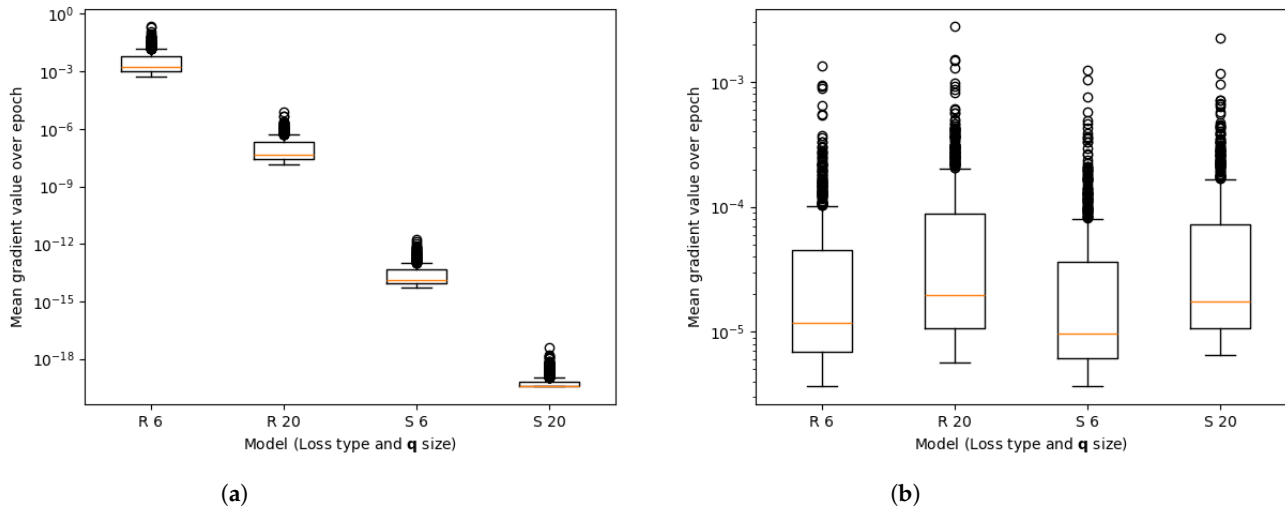


Figure 5. Boxplots showing the spread of the mean value per epoch of the applied gradients during training, with four different training strategies. (a) Shows the results when no rescaling of the loss is applied. (b) Shows the results when the rescaling factors $e_{\text{POD},d}$ and $e_{\text{POD},R}$ are applied. The latter case is invariant to the choice of latent space size, contrary to the first one.

We compare the relative snapshot error (e_u from Equation (46)) applied to the reconstructed snapshots $\mathbf{u}_j = D_u(E_u(\mathbf{u}_j^*))$ for the whole test dataset. We carry this out for the three models specified above, and for all the $\mathbf{q}_j$ sizes specified in Section 6. Then, this is performed again, but we take $\mathbf{u}_j$ as the result of the online ROM simulation with each architecture. The results are illustrated in Figure 6.

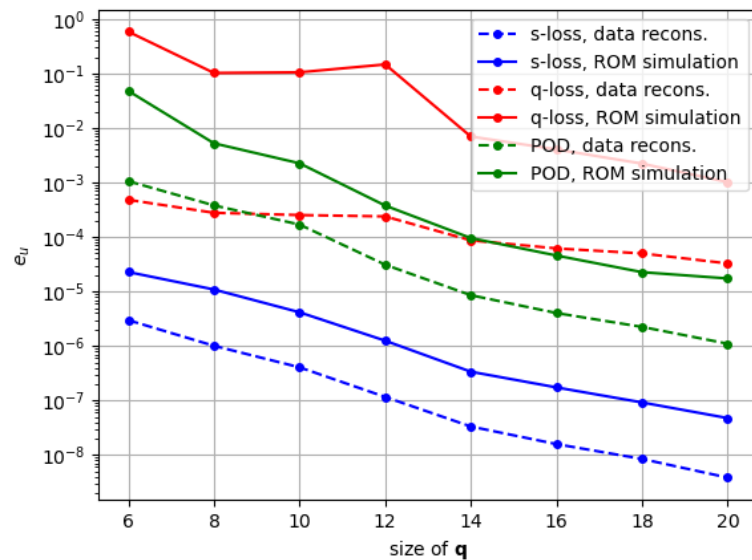


Figure 6. Comparison of the error in the snapshot, e_u , over three different architectures: (1) *s-loss* with our modified PROM-ANN method, (2) *q-loss* with the original PROM-ANN method in [32], and (3) *POD* with the traditional *POD* method. All of them form different dimensions of latent space. Dotted lines represent the error on data reconstruction alone, while full lines represent error on online ROM simulation. *s-loss* consistently performs several orders of magnitude better than *POD*, while *q-loss* is not able to keep up with *POD* in most cases.

For the data reconstruction, the error for our proposed methodology, in s -loss, is several orders of magnitude lower than both POD and q -loss, while the error for q -loss is marginally better than POD only for the lowest sizes of $\mathbf{q}$. We attribute this to the bad conditioning of the NN input, at least for problems with fast-decaying singular values like our use case. Without proper training, the secondary term of the PROM-ANN decoder will just add noise to the final snapshot, making it behave even worse than traditional POD .

These observations translate to a similar behavior in the ROM simulations. All the ROM simulations lose some accuracy compared to the data reconstruction, but this gap is even larger for the q -loss case.

7.2. Comparison of Snapshot and Residual Losses

The next step is to make a comparison of the effect of training based purely on data versus training on residuals. Like the subsection before, we compare models with $\mathbf{q}$ size ranging from 6 to 20.

Figure 7 compares errors on snapshots (e_u from Equation (46)). On the one hand, we check for the encoder–decoder data reconstruction and, on the other hand, for the results of the intrusive ROM execution. Meanwhile, Figure 8 compares the errors on residuals (e_R from Equation (47)) only for the encoder–decoder reconstruction.

In both figures, we compare two models that are trained differently:

- s -loss: Train using only the snapshot loss ($\mathcal{L}$ from Equation (44) with $\omega_d = 1$, $\omega_R = 0$) for 800 epochs, with the learning rate starting at 10^{-3} (same as in the previous subsection);
- r -loss: Start weights on the ones resulting from s -loss. Then, train for 800 epochs with only the residual loss ($\mathcal{L}$ from Equation (44) with $\omega_d = 0$, $\omega_R = 1$), with the learning rate starting at 10^{-4} .

The behavior represented in Figure 7 is quite interesting. Although the final result of the ROM simulation using the r -loss model is not substantially better than its s -loss counterpart, it consistently achieves a higher accuracy. This is in total opposition to the results for the encoder–decoder reconstruction of the snapshots, in which s -loss gets the highest accuracy for all $\mathbf{q}$ sizes. This seemingly contradictory phenomenon is clarified when looking at the reconstruction residuals in Figure 8, which shows the r -loss performing consistently better. These results hint that the behavior of the intrusive ROM may be more influenced by the correctness of the residual representation for the snapshots in our ROM approximation manifold than by the accuracy of the snapshots per se.

A small set of different architecture configurations in terms of neural network layers and training batch size was tested for their accuracy in online ROM simulation (see Appendix C). The results did not vary significantly among different architectures, while models fine-tuned on residual-based loss consistently behaved slightly better than their data-based counterparts. This hints towards the robustness of the methodology in terms of chosen hyper-parameters, at least for the particular use case.

For visualization purposes, Figure 9 shows the displacement once applied to the cantilever for a random test case $\mu = [2662.93, -1695.13]$. It does so with the results from FOM, traditional POD using $n = 6$, our PROM-ANN methodology with $n = 6$ and snapshot-based loss, and our PROM-ANN methodology with $n = 6$ and residual-based loss. As we can see, the two latter ones are virtually equal to the FOM representation, while traditional POD presents visual differences.

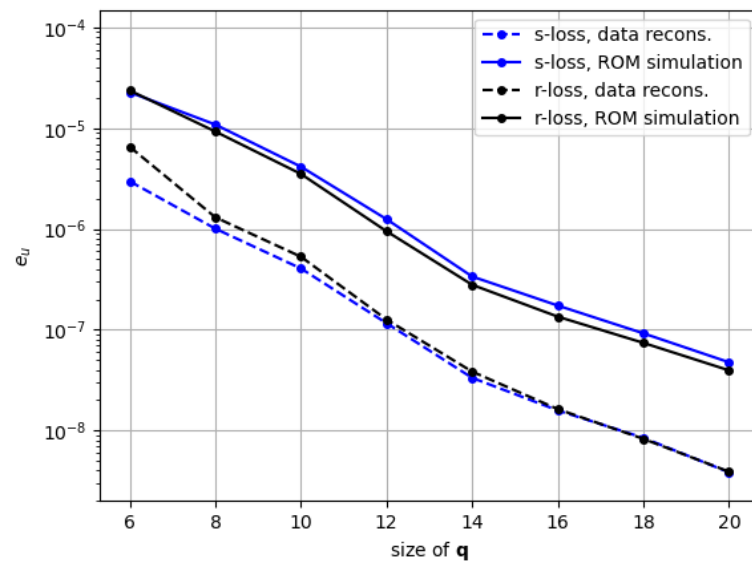


Figure 7. Comparison of the error in the snapshot, e_u , for models trained on loss $\mathcal{L}$ from Equation (44) with two different configurations: (1) s -loss with $(\omega_d = 1, \omega_R = 0)$ from randomly initialized weights and (2) r -loss with $(\omega_d = 0, \omega_R = 1)$ but starting from the weights of its s -loss counterpart. Both of them for different dimensions of latent space. Dotted lines represent error on data reconstruction alone, while full lines represent error on online ROM simulation. r -loss consistently gets slightly worse accuracy in the data reconstruction, while its ROM simulation solutions are slightly better than s -loss.

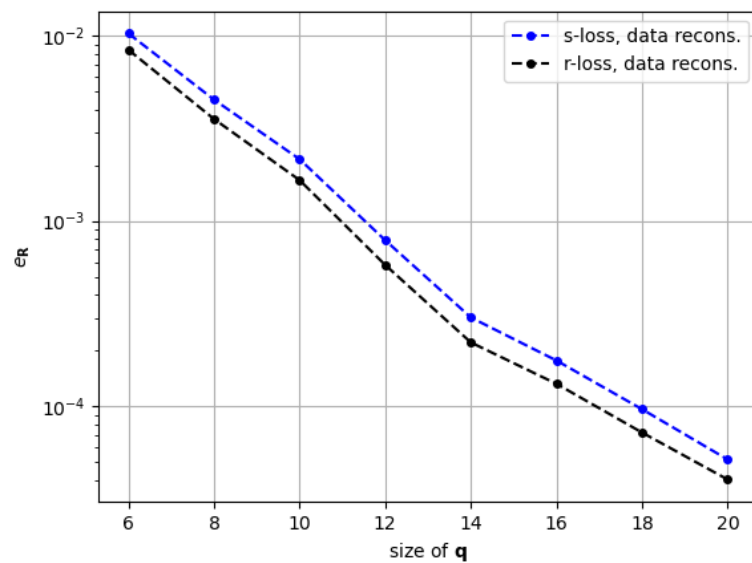


Figure 8. Comparison of the error in the residual, e_R , for models s -loss and r -loss as described in Figure 7. Both of these comparisons are for different dimensions of latent space. The error is computed only for the case of data reconstruction. r -loss consistently gets lower error than s -loss.

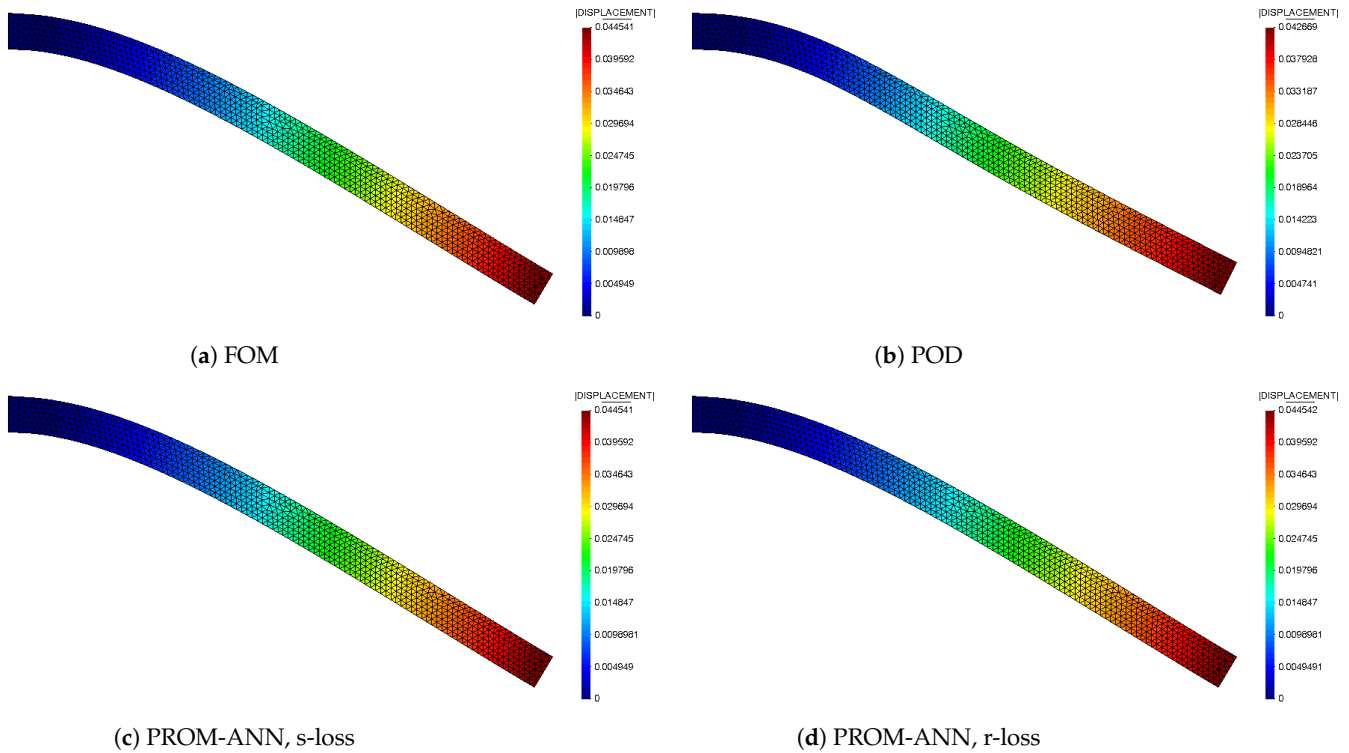


Figure 9. Displacement representation with the results from four different approaches for parameter vector $\mu = [2662.93, -1695.13]$. All ROM variants use $n = 6$. A slight visual difference is observable between POD and FOM, while the other two models are indistinguishable from FOM.

7.3. Comparison of Training and Online ROM Runtimes

For the training time, we check the mean time spent to train a single batch, measured over an entire epoch. As stated in Section 6, all cases share the same architecture (two hidden layer of size 200) and train with batches of size 16. Computation is performed on a single thread of an AMD Ryzen 7 5800x3d processor.

The description for the cases to compare is the following:

- Snapshot: Using loss $\mathcal{L}$ from Equation (44) with $\omega_d = 1$, $\omega_R = 0$. No interaction with FEM software.
- Residual (optimized): $\mathcal{L}$ from Equation (44) with $\omega_d = 0$, $\omega_R = 1$. Implements the gradients computation using the optimizations detailed in Equation (45).
- Residual (non-optimized): $\mathcal{L}$ from Equation (44) with $\omega_d = 0$, $\omega_R = 1$. Implements the gradients computation in a naive way by first computing $\mathbf{A}_i = (\mathbf{R}(\mathbf{u}_i) - \mathbf{R}(\mathbf{u}_i^*))^T \frac{\partial \mathbf{R}}{\partial \mathbf{u}}$ in KratosMultiphysics and then computing $\mathbf{B}_i = \frac{\partial \mathbf{u}_i}{\partial \Theta}$ via the `tf.GradientTape.batch_jacobian()` method in TensorFlow; then, it performs matrix multiplication in the following:

$$\frac{\partial \mathcal{L}_R}{\partial \Theta} = \frac{2}{e_{base,R}} \frac{1}{Nm} \sum_{i=1}^N \mathbf{A}_i \mathbf{B}_i \quad (48)$$

The results are shown in Table 1. We see that training via the residual with the naive implementation is not feasible, as the time for a single batch is in the order of seconds. Thanks to the optimizations described in Section 3.2, we avoid computing entire Jacobian matrices, which reduces the time to the order of 10 milliseconds. Still, as we expected, training the neural network via the residual loss takes considerably longer than training via the snapshot loss (around 40 times longer). That is why in Section 7.2 we perform a fine-tuning on the residual loss instead of training from random weights.

Table 1. Comparison of training time for a single batch among different losses.

Loss Type	Mean Batch Training Time (s)
Snapshot	9.20×10^{-4}
Residual (optimized)	3.87×10^{-2}
Residual (non-optimized)	7.41

Next, we want to make a performance assessment for the online ROM execution. For that, we measure the time that the FEM software takes to solve the reduced linear system of equations. Specifically, we take the mean resolution time for all the nonlinear iterations while solving nine different cases from our test set. This is performed for two different configurations of the architecture proposed in this paper, two cases of traditional POD that achieve similar accuracies to the former pair, and the FOM simulation. Again, these evaluations have been performed using a single thread from an AMD Ryzen 7 5800x3d processor.

The results for different models can be seen in Table 2.

Table 2. Comparison of computation times for various tasks within the online ROM routine over a set of different models.

Model	q Size	e_u	System Solve Time (s)
Modified PROM-ANN (Equation (33))	6	7.67×10^{-5}	1.42×10^{-6}
Modified PROM-ANN (Equation (33))	20	6.12×10^{-8}	5.55×10^{-6}
POD	18	2.39×10^{-5}	4.84×10^{-6}
POD	40	1.32×10^{-7}	1.99×10^{-5}
FOM	-	-	2.11×10^{-3}

As expected, the main computational advantage of our method (and of the original PROM-ANN [32], as both architectures are equivalent in terms of computational complexity) comes from the achieving smaller linear systems to solve for the same precision. It is worth pointing out that the biggest gains in computation time for the online process come with a further step in the ROM workflow called Hyper-Reduction, in which the amount of elements to take into account for the generation of the system at each step is greatly reduced. The number of reduced elements to use is directly related to the latent space dimension. Therefore, the reduction in modes that we achieve is also beneficial in this regard. This study focuses on the offline part of the ROM process, so there is no further development on the online procedure compared to the original PROM-ANN method. The implementation of Hyper-Reduction is described for this type of architecture in their paper [32], so interested readers are encouraged to check their performance assessments, which should be applicable to our case as well.

8. Discussion and Future Work

Once we have assessed the performance of our proposed architecture and losses, we will further discuss the implications of these results and the impact of our contributions.

We comment first on the discrete FEM residual-based loss that we developed throughout Section 3, and its proposed implementation strategy. These developments pose a step-up from recent initiatives to design discrete PINN-like losses [44,45,65,66] that limit themselves to linear problems of different natures. By taking advantage of open-source FEM software like KratosMultiphysics [58], we can access all the FEM methodology needed to obtain residuals and Jacobians for nonlinear cases in a wide range of state-of-the-art

FEM formulations. The cost for this is a loss in time efficiency during training compared to the classic data-based approach, because of the required dynamic interaction between the FEM software and the neural network framework during training and also the manual computation of the gradient loss. It is in this sense that our implementation proposal makes a huge difference, taking the training time from being prohibitive to being just an order of magnitude higher than the data-based one. These comparisons are shown in Section 7.3. We believe that the main computational bottleneck after our proposed methodology is the fact that the FEM software currently runs the computations in series for each sample in the batch. Some future work on parallelizing these procedures could potentially unlock training times much closer to the data-based ones. In addition, the current formulation introduces a fundamental architectural shift compared to the original PROM-ANN framework [32], which operated entirely in the reduced-order space, learning a map from $q \in \mathbb{R}^n$ to $\bar{q} \in \mathbb{R}^{\bar{n}}$. In contrast, our physics-aware variant requires training in the high-dimensional physical space of the full-order model, since the FEM residual and its Jacobian must be evaluated in that space. This change increases training cost considerably, both due to the FEM evaluations and the need to retrieve full Jacobians for backpropagation. While this enables the integration of high-fidelity physics, it does not scale well for large-scale problems. Future work could address this by coupling the current approach with scalable network architectures—e.g., convolutional or graph-based neural networks—and by projecting the residual into an intermediate reduced-order space before backpropagation. Related ideas have emerged in the recent literature, notably in the form of semi-intrusive training strategies such as the one proposed by Halder et al. [72], where residuals are used only in projected form during training, avoiding full-order evaluations. However, these approaches preserve non-intrusiveness. We believe such hybrid strategies are promising and plan to explore them in future developments. Another key characteristic of the proposed loss is being parameter-agnostic. This makes it more versatile in various ways: in terms of efficiency, the FEM software does not need to re-configure the simulation for each specific sample, and in terms of use case, it can be applied to cases in which not only the minimization of the residual itself is important, but also its behavior while being non-zero. This latter aspect is key in using this loss for our particular setting of intrusive ROM. One unexplored advantage of this loss formulation would be the possibility to perform partial physics-based learning, where only specific components of the total residual are used for the training. For example, one could train only on the steady-state component of the residual of a dynamic case in order to avoid the inconveniences from the dependency on previous time-steps. This is not addressed in this paper, but left as possible future contributions. There are further options that the residual loss could open up and that we have not fully explored, e.g., the possibility of training on the residual with noisy data in order to achieve data augmentation. Next, we comment on the modification to the PROM-ANN architecture itself and the data-based loss $\mathcal{L}_d$ with the scaling matrices Ξ , $\tilde{\Xi}$, and the global scaling $1/e_{\text{POD},d}$. The scaling matrices are an inexpensive way to normalize the input ranges for the neural network with the direct results from the SVD so that we avoid extra statistical studies of the dataset. The global scaling is a single scalar computed inexpensively via POD, only once for the whole training. The effect of these modifications is apparent in the results in Section 7.1, where both the reconstruction and ROM results from our architecture are several orders of magnitude better than the simpler, original approach described in [32]. Now there is a very plausible explanation for the lack of performance of the original PROM-ANN, mostly when comparing with the good results that they obtain in their paper. The use case that they use for evaluation is a 2D inviscid Burgers problem, which has a much flatter decay in the SVD's singular values compared to ours. Thus, the range of their inputs to the neural network should

naturally be more uniform. Another possibility is that they apply some normalization routine prior to the neural network without mentioning it explicitly. In any case, we can say that our modification makes the architecture generalizable to any kind of problem in terms of their singular value decay. Additionally, the global scaling $1/e_{\text{POD},d}$ is key in order to stabilize the scale of the loss and the backpropagation gradients, making the neural network optimizer perform equally whatever the choice of latent size. It also has an interpretational purpose, which is to make the loss a direct indicator of how much better the results are relative to the simple POD version. Something to explore in the future would be better methodologies to choose the modes included in the primary and secondary ROBs. Right now, this is performed in a greedy way, selecting as many modes that are needed to achieve a certain accuracy in POD, but it is not clear how different modes are related to one another (especially since they are uncorrelated in the linear sense). The implementation of our version of ANN-PROM (only with the data-based loss) is readily available within the ROM application of KratosMultiphysics [58]. The framework to train via residual has not yet been implemented in the master branch of the KratosMultiphysics software at the time of publication. Until it is fully implemented, interested users can check out the branch at https://github.com/KratosMultiphysics/Kratos/tree/RomApp_RomManager_ResidualTrainingStructural (accessed on 15 May 2025) which handles the residual usage for the StructuralMechanicsApplication. Once using that branch, the user may run the example in https://github.com/KratosMultiphysics/Examples/tree/master/rom_application/RomManager_cantilever_NN_residual (accessed on 15 May 2025) which implements this cantilever use case (with a reduced number of samples by default).

Finally, we discuss the effect of training our modified ANN-PROM architecture on the residual loss. Other than the increased duration of the training routine, it was also difficult for us to prevent the model from falling in non-optimal local minima during training with the residual loss. This is why the chosen approach was to train first with the data-based approach and only then fine-tune with a purely physics-based loss. The intuition for proposing this physics-based training comes from the fact that the intrusive ROM accuracy is not only given by the snapshot-reconstruction capabilities of the approximation manifold (which would work more like a lower limit in terms of error), but should also be dependant on how well the residuals are represented within these solutions in the manifold. This is essentially because the residual is the quantity being optimized during the intrusive ROM simulation. The general discrepancy between ROM and reconstruction is clearly demonstrated within our use case in Section 7.1, with the error in ROM simulation being approximately one order of magnitude higher than the reconstruction one for both our proposed architecture with the data-based loss and traditional POD. Further on, we look at the results in Section 7.2 to specifically understand the effect of the residual-based training. The observations are encouraging, even if not spectacular. We say this in the sense that performing ROM with the model trained on the residual provided slightly but consistently better results than the one trained with the snapshot. It can also be interpreted as achieving a slightly lower discrepancy between reconstruction and ROM simulation, which is what we were aiming for. But the most important insight is that this phenomenon coincides with a consistently lower error in the representation of the residual by the models trained on physics. We are aware that, as it is right now, the significantly higher training time with the residual loss renders the proposed method unattractive, taking into account the marginal increase in ROM accuracy, but we are optimistic that future research can enable more meaningful improvements.

We make the observation, in retrospective, that choosing the ANN-PROM architecture for implementing the residual loss limited the achievable accuracy. That is because, even if the neural network allows us to introduce the loss of our choosing, we still depend entirely on the modes that we gathered via SVD on our snapshots dataset, without any regard for the residual accuracy. The fact that even with this caveat we were able to achieve slightly better results makes us very optimistic for the future where we could explore new architectures (or variants of this one) that put the residual in the focus point from the beginning, or that allow more freedom to correct the residual on top of the modes for the snapshot. This restriction of ANN-PROM towards the flexibility of the residual also hinders the potential ability of the residual training to enhance the model's behavior outside the training parameter space, which is a topic worthy of studying in detail in further work (see Appendix B).

Although the present work focuses solely on augmenting the PROM-ANN framework with a discrete residual loss to enhance physics-awareness, it is useful to briefly comment on its positioning within the broader landscape of nonlinear ROMs. In this paper, comparisons were limited to traditional linear subspace PROMs, as our contribution centers specifically on the integration of the high-fidelity residual in the PROM-ANN training. Broader comparisons with alternative nonlinear ROM techniques—such as quadratic ROMs or kriging-based interpolation—relate more directly to the PROM-ANN methodology itself, independently of the residual loss term. In this context, recent studies such as [33] have benchmarked PROM-ANN against several nonlinear ROM strategies. Furthermore, the residual-based loss proposed in this work is not restricted to PROM-ANN and can be readily integrated into other neural network-based ROM architectures, such as convolutional autoencoder ROMs [28]. It is also fair to note that, within the broader family of nonlinear PROMs, our formulation remains fully compatible with local or piecewise PROM-ANN variants—whether using a single ANN that adapts to the active local basis, or multiple models trained for different regions in parameter space. These broader methodological directions are considered valuable future extensions.

All in all, the three main contributions of the paper work together to complement each other and obtain a physics-informed intrusive ROM framework, but also hold value by themselves: the residual loss could be applied for other purposes like non-intrusive ROM, the modifications to the ANN-PROM architecture make it more versatile in terms of the types of problems it can handle, and the study of the effect of the residual in intrusive ROM provides a seed for a new path of research in this discipline for us and for the rest of the community in numerical methods.

9. Conclusions

In this paper, we extend the PROM-ANN architecture proposed in [32] by incorporating a training approach based on the finite element method (FEM) residual, rather than relying solely on snapshot data. This establishes a connection between nonlinear reduced-order models (ROMs) and physics-informed neural networks (PINNs). While traditional PINNs use analytical partial differential equations (PDEs) to train continuous, non-intrusive models, our approach leverages discrete FEM residuals as the loss function for backpropagation, guiding the learning of the ROM approximation manifold. This development allows us to investigate the impact of improving the residual of the snapshots on the overall performance of projection-based ROMs.

The path to achieve this final goal enables us to present three independently significant contributions: (1) a loss based on the FEM residual and which is parameter-agnostic and, most importantly, applicable to nonlinear problems; (2) a modification on the original PROM-ANN architecture in [32] that makes it applicable to cases with fast-decaying

singular values, and (3) a study on the effect of the residual-based training on ROM simulation. We demonstrate our approach in the context of static structural mechanics with nonlinear hyperelasticity, specifically in the deformation of a rubber cantilever subjected to two orthogonal variable loads.

In terms of the residual loss, the fact that it is based on existing FEM software makes it applicable to a high range of problems. The proposed implementation strategy makes the interaction between the neural network and the FEM software viable in reasonable training time, around 40ms per batch. Finally, the enhancement of the resulting residuals via this loss is demonstrated by applying it to our proposed PROM-ANN-based architecture and performing data reconstruction. This results in a consistently lower residual representation compared to the cases trained on snapshot data alone.

In terms of the modifications to the original ANN-PROM architecture, we observe how our method lowers the snapshot error by several orders of magnitude compared to POD in both the data reconstruction and the ROM simulation results. This improvement is consistent for latent space sizes ranging from $n = 6$ to $n = 20$. In contrast, the original PROM-ANN formulation struggles to train the neural network, resulting in errors higher than POD in most cases. This enhancement from our methodology comes from a proper scaling strategy on both the architecture itself and the loss that is used.

Finally, regarding the effect of applying the FEM residual loss on the approximation manifold for projection-based ROM, our results show a modest but consistent reduction of the gap between the accuracy for snapshot reconstruction and the accuracy for projection-based ROM. While not being useful in practice as of now, this observation makes us optimistic that better results for projection-based ROM could be unlocked in the future by taking care of the residual representations within the approximation manifold. In the future, alternative nonlinear ROM architectures that enable more control over the resulting solutions' residuals can greatly improve the results presented in this work.

As a corollary to the discussion and to provide a roadmap for future work, we summarize below, in Table 3, the main current limitations of the proposed approach and potential directions to address them:

Table 3. Summary of current limitations and future research directions for the proposed discrete physics-informed residual loss for projection-based ROMs.

Current Limitation	Potential Direction
High training cost due to serial FEM residual evaluations.	Parallelize FEM evaluations across mini-batches to improve training throughput.
Residual evaluations are performed in full-order space, limiting scalability.	Project residuals into intermediate reduced-order spaces to reduce computational cost.
Residual-based training yields only modest gains in ROM accuracy. Attributed to PROM-ANN being limited by fixed SVD-based modes and not being able to adapt to residual structure.	Design residual-aware or adaptive mode selection strategies. Architectures like convolutional or graph neural networks may be good candidates for their flexibility, but may be limited in terms of scalability.
Extrapolation remains limited due to a lack of residual-focused manifold generalization.	Explore local or multi-network manifold strategies; study residual-informed extrapolation systematically.
Method limited to forward problems.	Explore inverse problems via adjoint-based gradients or parameter-to-output mappings.

Author Contributions: Conceptualization, R.R. and N.S.; methodology, N.S., S.A.d.P. and J.R.B.; software, N.S., S.A.d.P. and J.R.B.; validation, N.S.; formal analysis, N.S.; investigation, N.S.; resources, R.R.; data curation, N.S.; writing—original draft preparation, N.S., S.A.d.P. and J.R.B.; writing—review and editing, N.S., S.A.d.P. and J.R.B.; visualization, N.S.; supervision, R.R.; project administration, R.R.; funding acquisition, R.R. All authors have read and agreed to the published version of the manuscript.

Funding: N.S. acknowledges the Secretariat of Universities and Research of the Department of Research and Universities of the Generalitat of Catalonia, as well as the European Social Plus Fund for their financial support through the predoctoral scholarship AGAUR-FI (2024 FI-1 00089) Joan Oró. S.A.d.P. and J.R.B. acknowledge the Departament de Recerca i Universitats de la Generalitat de Catalunya for the financial support through the FI-SDUR 2020 and FI-SDUR 2021 scholarships. S.A.d.P. also acknowledges support from the Fulbright Commission Spain through the Fulbright Predoctoral Research Fellowship (2024–2025).

Data Availability Statement: The files to replicate the hyperelastic cantilever use-case can be found in https://github.com/KratosMultiphysics/Examples/tree/master/rom_application/RomManager_cantilever_NN_residual, (accessed on 15 May 2025). The same repository contains the script to both generate data for this case and train PROM-ANN instances via snapshot and residual. In order to run it, the user needs to install the specific branch of KratosMultiphysics [58] found in https://github.com/KratosMultiphysics/Kratos/tree/RomApp_RomManager_ResidualTrainingStructural (accessed on 15 May 2025). Further details and data can be provided on request to the authors.

Conflicts of Interest: The authors declare no conflicts of interest.

Nomenclature

HFM	High-fidelity model;
ROM	Reduced-order model;
POD	Proper orthogonal decomposition;
SVD	Singular value decomposition;
PINN	Physics-informed neural network;
PDE	Partial differential equation;
FEM	Finite elements method;
$\mathcal{N} : (a; \Theta) \mapsto b$	A neural network parametrized by Θ ;
u	Nodal solution of FEM problem;
q	FEM solution representation in a given reduced space;
$D : q \mapsto u$	A decoder function;
$E : u \mapsto q$	An encoder function;
μ	Parameters vector for a FEM simulation;
$\mathbf{R}(u; \mu)$	FEM residual, given nodal solution u and simulation parameters μ ;
$\mathcal{L}(\Theta)$	Loss function used to optimize parameters set Θ ;
$N \in \mathbb{N}$	Number of degrees of freedom in the FOM;
$n \in \mathbb{N}$	Number of degrees of freedom in ROM. Number of POD modes. Latent space dimensions;
$M \in \mathbb{N}$	Number of samples in the training dataset;
$m \in \mathbb{N}$	Number of samples in a given batch when training a neural network.

Appendix A. Reduced Dataset

This appendix shows the details for the performance of our methodology while using a reduced training dataset of 500 FOM samples. The compared models are those from Section 7.2 with 6, 10, 14, and 18 primary modes, which were trained with 5000 samples, and their equivalent counterparts trained in the reduced dataset. Figure A1 shows how the reduced dataset leads to a moderate degradation in accuracy, but the models are still significantly more accurate than the POD baseline, both with and without the proposed

residual-based training. This confirms that, for many applications, 500 samples may already offer a satisfactory trade-off between offline cost and prediction quality.

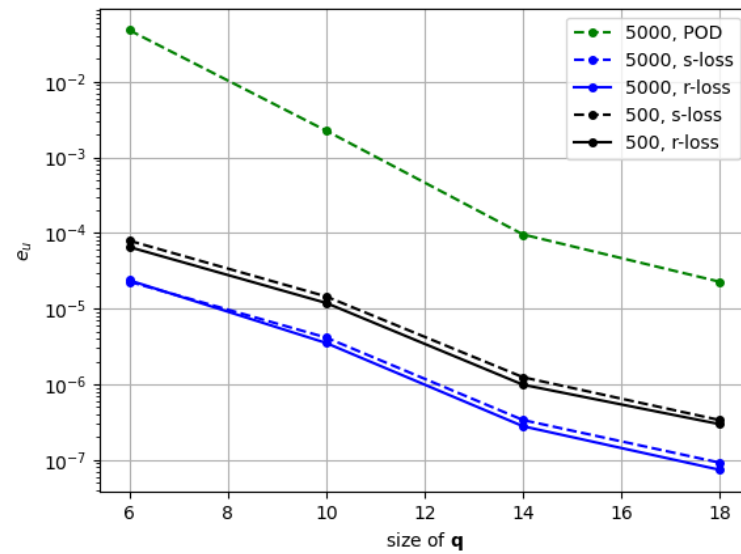


Figure A1. Comparison of the relative error e_u versus the reduced coordinate $\mathbf{q}$ size for POD and PROM-ANN variants trained with 500 and 5000 FOM simulations. PROM-ANN consistently outperforms POD, and residual-based training leads to improved accuracy in both sample regimes.

Appendix B. Extrapolation Ability

This appendix shows the results of testing models resulting from our methodology outside of the training parameter space. Specifically, we generate a test set of 50 FOM samples outside the parametric space, within 500 N/m off the limits. Then, we test the same models as in Section 7.2 (for 6, 10, 14, and 18 primary modes) on this new test set and compare them to the results on the original test dataset. Figure A2 shows how, while significantly inferior to interpolation capabilities, when using a higher number of primary modes the capacity for extrapolation becomes reasonable, achieving errors lower than 10^{-3} .

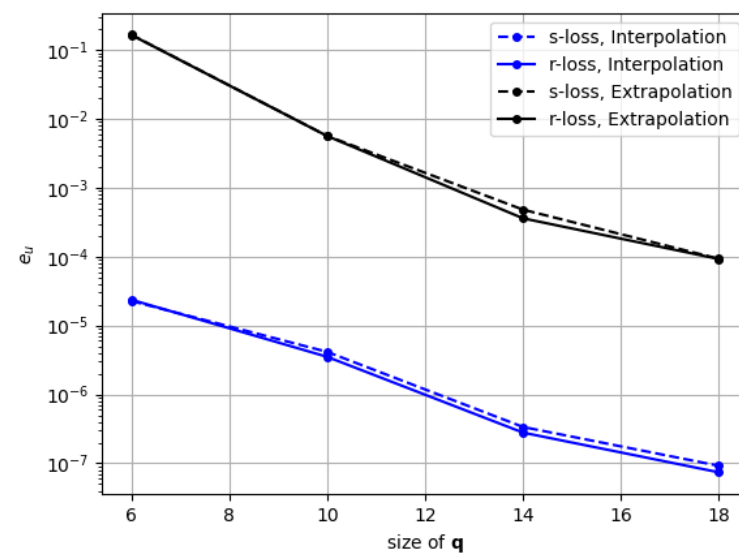


Figure A2. Comparison of the relative error e_u versus the reduced coordinate $\mathbf{q}$ size for PROM-ANN variants evaluated either in (1) the original test set (interpolation): all samples contained within the parametric space, or (2) an extrapolation set: 50 samples outside the parametric space, within 500 N/m of the limits. While the accuracy is severely decreased compared to interpolation, the extrapolation performance is still reasonable for the cases with higher $\mathbf{q}$ size.

Appendix C. Robustness to Hyper-Parameter Choice

This last appendix shows the results of training a few extra ANN-PROM models with different neural network layer configurations or batch sizes. All of these new models use $n = 14$ and $\bar{n} = 46$, and both the snapshot-based and residual-based models are represented. They have all been evaluated for online snapshot accuracy e_u .

The gathered accuracies, listed in Table A1, do not vary significantly from one architecture to another. On top of this, models fine-tuned on the residual-based loss still consistently have a slight advantage over their data-based trained peers. Therefore, the results indicate that the proposed methodology is indeed applicable to various configurations, at least in the use case at hand.

Table A1. Comparative study of different neural network architecture configurations (all with $n = 14$ and $\bar{n} = 46$). The relative error on snapshot e_u from online ANN-PROM simulation is shown for models trained on data only (s-loss) and models fine-tuned on residual loss (r-loss). Performance does not change significantly among different architectures, but r-loss models still consistently have a small advantage over s-loss.

Layers Size	Batch Size	e_u (s-Loss)	e_u (r-Loss)
200	16	7.43×10^{-7}	6.49×10^{-7}
200, 200	8	4.31×10^{-7}	3.24×10^{-7}
200, 200	16	3.37×10^{-7}	2.79×10^{-7}
400, 400	16	4.09×10^{-7}	3.26×10^{-7}

References

1. Brunton, S.L.; Kutz, J.N.; Manohar, K.; Aravkin, A.Y.; Morgansen, K.; Klemisch, J.; Goebel, N.; Buttrick, J.; Poskin, J.; Blom-Schieber, A.W.; et al. Data-driven aerospace engineering: Reframing the industry with machine learning. *AIAA J.* **2021**, *59*, 2820–2847. [\[CrossRef\]](#)
2. Jiang, Y.; Yin, S.; Li, K.; Luo, H.; Kaynak, O. Industrial applications of digital twins. *Philos. Trans. R. Soc. A* **2021**, *379*, 20200360. [\[CrossRef\]](#) [\[PubMed\]](#)
3. de Parga, S.A.; Bravo, J.R.; Sibuet, N.; Hernandez, J.A.; Rossi, R.; Boschert, S.; Quintana-Ortí, E.S.; Tomás, A.E.; Tatu, C.C.; Vázquez-Novoa, F.; et al. Parallel reduced order modeling for digital twins using high-performance computing workflows. *arXiv* **2024**, arXiv:2409.09080.
4. Hesthaven, J.S.; Rozza, G.; Stamm, B. *Certified Reduced Basis Methods for Parametrized Partial Differential Equations*; SpringerBriefs in Mathematics; Springer: Cham, Switzerland, 2016.
5. Quarteroni, A.; Rozza, G. *Reduced Order Methods for Modeling and Computational Reduction*; Modeling, Simulation and Applications; Springer: Cham, Switzerland, 2014; Volume 9.
6. Carlberg, K.; Bou-Mosleh, C.; Farhat, C. Efficient non-linear model reduction via a least-squares petrov-galerkin projection and compressive tensor approximations. *Int. J. Numer. Methods Eng.* **2011**, *86*, 155–181. [\[CrossRef\]](#)
7. Carlberg, K.; Farhat, C.; Cortial, J.; Amsallem, D. The gnat method for nonlinear model reduction: Effective implementation and application to computational fluid dynamics and turbulent flows. *J. Comput. Phys.* **2013**, *242*, 623–647. [\[CrossRef\]](#)
8. Carlberg, K.; Barone, M.; Antil, H. Galerkin v. least-squares petrov-galerkin projection in nonlinear model reduction. *J. Comput. Phys.* **2017**, *330*, 693–734. [\[CrossRef\]](#)
9. Grimberg, S.; Farhat, C.; Tezaur, R.; Bou-Mosleh, C. Mesh sampling and weighting for the hyperreduction of nonlinear petrov-galerkin reduced-order models with local reduced-order bases. *Int. J. Numer. Methods Eng.* **2021**, *122*, 1846–1874. [\[CrossRef\]](#)
10. de Parga, S.A.; Bravo, J.R.; Hernández, J.A.; Zorrilla, R.; Rossi, R. Hyper-reduction for petrov-galerkin reduced order models. *Comput. Methods Appl. Mech. Eng.* **2023**, *416*, 116298. [\[CrossRef\]](#)
11. Hesthaven, J.S.; Ubbiali, S. Non-intrusive reduced order modeling of nonlinear problems using neural networks. *J. Comput. Phys.* **2018**, *363*, 55–78. [\[CrossRef\]](#)
12. Wang, Q.; Hesthaven, J.S.; Ray, D. Non-intrusive reduced order modeling of unsteady flows using artificial neural networks with application to a combustion problem. *J. Comput. Phys.* **2019**, *384*, 289–307. [\[CrossRef\]](#)
13. Guo, M.; Hesthaven, J.S. Reduced order modeling for nonlinear structural analysis using gaussian process regression. *Comput. Methods Appl. Mech. Eng.* **2018**, *341*, 807–826. [\[CrossRef\]](#)

14. Guo, M.; Hesthaven, J.S. Data-driven reduced order modeling for time-dependent problems. *Comput. Methods Appl. Mech. Eng.* **2019**, *345*, 75–99. [\[CrossRef\]](#)
15. Casenave, F.; Ern, A.; Lelièvre, T. A nonintrusive reduced basis method applied to aeroacoustic simulations. *Adv. Comput. Math.* **2015**, *41*, 961–986. [\[CrossRef\]](#)
16. Sirovich, L. Turbulence and the dynamics of coherent structures. part I: Coherent structures. *Q. Appl. Math.* **1987**, *45*, 561–571. [\[CrossRef\]](#)
17. Holmes, P.; Lumley, J.L.; Berkooz, G. *Turbulence, Coherent Structures, Dynamical Systems and Symmetry*; Cambridge Monographs on Mechanics; Cambridge University Press: Cambridge, UK, 1996.
18. Eckart, C.; Young, G. The approximation of one matrix by another of lower rank. *Psychometrika* **1936**, *1*, 211–218. [\[CrossRef\]](#)
19. Antoulas, A.C. An overview of approximation methods for large-scale dynamical systems. *Annu. Rev. Control* **2005**, *29*, 181–190. [\[CrossRef\]](#)
20. Cuong, N.N.; Veroy, K.; Patera, A.T. *Certified Real-Time Solution of Parametrized Partial Differential Equations*; Springer: Dordrecht, The Netherlands, 2005.
21. Rowley, C.W.; Colonius, T.; Murray, R.M. Model reduction for compressible flows using pod and galerkin projection. *Phys. D Nonlinear Phenom.* **2004**, *189*, 115–129. [\[CrossRef\]](#)
22. Benner, P.; Gugercin, S.; Willcox, K. A survey of projection-based model reduction methods for parametric dynamical systems. *SIAM Rev.* **2015**, *57*, 483–531. [\[CrossRef\]](#)
23. Xu, J.; Duraisamy, K. Multi-level convolutional autoencoder networks for parametric prediction of spatio-temporal dynamics. *Comput. Methods Appl. Mech. Eng.* **2020**, *372*, 113379. [\[CrossRef\]](#)
24. Pichi, F.; Moya, B.; Hesthaven, J.S. A graph convolutional autoencoder approach to model order reduction for parametrized PDEs. *arXiv* **2023**, arXiv:2305.08573. [\[CrossRef\]](#)
25. Iollo, A.; Lanteri, S.; Désidéri, J.-A. Stability properties of pod–galerkin approximations for the compressible navier–stokes equations. *Theor. Comput. Fluid Dyn.* **2000**, *13*, 377–396. [\[CrossRef\]](#)
26. Huang, C.; Duraisamy, K.; Merkle, C. Challenges in reduced order modeling of reacting flows. In Proceedings of the 2018 Joint Propulsion Conference, Cincinnati, OH, USA, 9–11 July 2018; p. 4675.
27. Pinkus, A. *n-Widths in Approximation Theory*; Springer: Berlin/Heidelberg, Germany, 1985.
28. Lee, K.; Carlberg, K.T. Model reduction of dynamical systems on nonlinear manifolds using deep convolutional autoencoders. *J. Comput. Phys.* **2020**, *404*, 108973. [\[CrossRef\]](#)
29. Barnett, J.; Farhat, C. Quadratic approximation manifold for mitigating the kolmogorov barrier in nonlinear projection-based model order reduction. *J. Comput. Phys.* **2022**, *464*, 111348. [\[CrossRef\]](#)
30. Amsallem, D.; Zahr, M.J.; Farhat, C. Nonlinear model order reduction based on local reduced-order bases. *Int. J. Numer. Methods Eng.* **2012**, *92*, 891–916. [\[CrossRef\]](#)
31. Bravo, J.R.; Hernández, J.A.; de Parga, S.A.; Rossi, R. A subspace-adaptive weights cubature method with application to the local hyperreduction of parameterized finite element models. *Int. J. Numer. Methods Eng.* **2024**, *125*, e7590. [\[CrossRef\]](#)
32. Barnett, J.; Farhat, C.; Maday, Y. Neural-network-augmented projection-based model order reduction for mitigating the kolmogorov barrier to reducibility. *J. Comput. Phys.* **2023**, *492*, 112420. [\[CrossRef\]](#)
33. Chmiel, M.R.; Barnett, J.L.; Farhat, C. Assessment of projection-based model order reduction for a benchmark hypersonic flow problem. In Proceedings of the AIAA SciTech 2024 Forum, Orlando, FL, USA, 8–12 January 2024; p. 0250.
34. Li, M.; Thurnher, T.; Xu, Z.; Jain, S. Data-free non-intrusive model reduction for nonlinear finite element models via spectral submanifolds. *Comput. Methods Appl. Mech. Eng.* **2025**, *434*, 117590. [\[CrossRef\]](#)
35. Cenedese, M.; Axås, J.; Bäuerlein, B.; Avila, K.; Haller, G. Data-driven modeling and prediction of non-linearizable dynamics via spectral submanifolds. *Nat. Commun.* **2022**, *13*, 872. [\[CrossRef\]](#)
36. Haller, G.; Ponsioen, S. Nonlinear normal modes and spectral submanifolds: Existence, uniqueness and use in model reduction. *Nonlinear Dyn.* **2016**, *86*, 1493–1534. [\[CrossRef\]](#)
37. Raissi, M.; Perdikaris, P.; Karniadakis, G.E. Physics informed deep learning (part i): Data-driven solutions of nonlinear partial differential equations. *arXiv* **2017**, arXiv:1711.10561.
38. Cuomo, S.; Cola, V.S.D.; Giampaolo, F.; Rozza, G.; Raissi, M.; Piccialli, F. Scientific machine learning through physics-informed neural networks: Where we are and what's next. *J. Sci. Comput.* **2022**, *92*, 88. [\[CrossRef\]](#)
39. Li, Z.; Zheng, H.; Kovachki, N.; Jin, D.; Chen, H.; Liu, B.; Azizzadenesheli, K.; Anandkumar, A. Physics-informed neural operator for learning partial differential equations. *ACM/IMS J. Data Sci.* **2024**, *1*, 1–27. [\[CrossRef\]](#)
40. Wang, S.; Yu, X.; Perdikaris, P. When and why pinns fail to train: A neural tangent kernel perspective. *J. Comput. Phys.* **2022**, *449*, 110768. [\[CrossRef\]](#)
41. Fuks, O.; Tchelepi, H.A. Limitations of physics informed machine learning for nonlinear two-phase transport in porous media. *J. Mach. Learn. Model. Comput.* **2020**, *1*, 19–37. [\[CrossRef\]](#)

42. Niu, S.; Zhang, E.; Bazilevs, Y.; Srivastava, V. Modeling finite-strain plasticity using physics-informed neural network and assessment of the network performance. *J. Mech. Phys. Solids* **2023**, *172*, 105177. [\[CrossRef\]](#)
43. Zienkiewicz, O.C. *The Finite Element Method in Engineering Science*; McGraw-Hill European Publishing Programme; McGraw-Hill: New York, NY, USA, 1971.
44. Meethal, R.E.; Kodakkal, A.; Khalil, M.; Ghantasala, A.; Obst, B.; Bletzinger, K.U.; Wüchner, R. Finite element method-enhanced neural network for forward and inverse problems. *Adv. Model. Simul. Eng. Sci.* **2023**, *10*, 6. [\[CrossRef\]](#)
45. Le-Duc, T.; Nguyen-Xuan, H.; Lee, J. A finite-element-informed neural network for parametric simulation in structural mechanics. *Finite Elem. Anal. Des.* **2023**, *217*, 103904. [\[CrossRef\]](#)
46. Eshaghi, M.S.; Anitescu, C.; Thombre, M.; Wang, Y.; Zhuang, X.; Rabczuk, T. Variational physics-informed neural operator (VINO) for solving partial differential equations. *Comput. Methods Appl. Mech. Eng.* **2025**, *437*, 117785. [\[CrossRef\]](#)
47. Wang, S.; Wang, H.; Perdikaris, P. Learning the solution operator of parametric partial differential equations with physics-informed deepnets. *Sci. Adv.* **2021**, *7*, eabi8605. [\[CrossRef\]](#)
48. Chen, W.; Wang, Q.; Hesthaven, J.S.; Zhang, C. Physics-informed machine learning for reduced-order modeling of nonlinear problems. *J. Comput. Phys.* **2021**, *446*, 110666. [\[CrossRef\]](#)
49. Hijazi, S.; Freitag, M.; Landwehr, N. Pod-galerkin reduced order models and physics-informed neural networks for solving inverse problems for the navier–stokes equations. *Adv. Model. Simul. Eng. Sci.* **2023**, *10*, 5. [\[CrossRef\]](#)
50. Brivio, S.; Fresca, S.; Manzoni, A. Ptpi-dl-roms: Pre-trained physics-informed deep learning-based reduced order models for nonlinear parametrized pdes. *Comput. Methods Appl. Mech. Eng.* **2024**, *432*, 117404. [\[CrossRef\]](#)
51. Sirignano, J.; Spiliopoulos, K. Dgm: A deep learning algorithm for solving partial differential equations. *J. Comput. Phys.* **2018**, *375*, 1339–1364. [\[CrossRef\]](#)
52. Eivazi, H.; Tahani, M.; Schlatter, P.; Vinuesa, R. Physics-informed neural networks for solving reynolds-averaged navier–stokes equations. *Phys. Fluids* **2022**, *34*, 075117. [\[CrossRef\]](#)
53. Kim, Y.; Choi, Y.; Widemann, D.; Zohdi, T. A fast and accurate physics-informed neural network reduced order model with shallow masked autoencoder. *J. Comput. Phys.* **2022**, *451*, 110841. [\[CrossRef\]](#)
54. As’Ad, F.; Avery, P.; Farhat, C.; Rabinovitch, J.; Lobbia, M. A mechanics-informed artificial neural network approach in data-driven constitutive modeling. *Int. J. Numer. Methods Eng.* **2022**, *123*, 2738–2759. [\[CrossRef\]](#)
55. Kelly, S.T.; Epureanu, B.I. Physics-informed machine learning approach for reduced-order modeling of integrally bladed rotors: Theory and application. *J. Sound Vib.* **2025**, *596*, 118773. [\[CrossRef\]](#)
56. Yang, L.-H.; Luo, X.-L.; Yang, Z.-B.; Nan, C.-F.; Chen, X.-F.; Sun, Y. Fe reduced-order model-informed neural operator for structural dynamic response prediction. *Neural Netw.* **2025**, *188*, 107437. [\[CrossRef\]](#)
57. Hong, Y.; Bansal, H.; Veroy, K. Physics-informed two-tier neural network for non-linear model order reduction. *Adv. Model. Simul. Eng. Sci.* **2024**, *11*, 20. [\[CrossRef\]](#)
58. Ferrándiz, V.M.; Bucher, P.; Zorrilla, R.; Warnakulasuriya, S.; Cornejo, A.; Rossi, R.; Roig, C.; Maria, J.; Masó, M.; Casas, G.; et al. Kratosmultiphysics/kratos: v10.1, November 2024. Available online: <https://zenodo.org/records/14185721> (accessed on 15 May 2025).
59. Brigham, J.C.; Aquino, W. Inverse viscoelastic material characterization using pod reduced-order modeling in acoustic–structure interaction. *Comput. Methods Appl. Mech. Eng.* **2009**, *198*, 893–903. [\[CrossRef\]](#)
60. Frangos, M.; Marzouk, Y.; Willcox, K.; van Bloemen Waanders, B. Surrogate and reduced-order modeling: A comparison of approaches for large-scale statistical inverse problems. In *Large-Scale Inverse Problems and Quantification of Uncertainty*; Wiley Online Library: Hoboken, NJ, USA, 2010; pp. 123–149.
61. Cai, S.; Mao, Z.; Wang, Z.; Yin, M.; Karniadakis, G.E. Physics-informed neural networks (pinns) for fluid mechanics: A review. *Acta Mech. Sin.* **2021**, *37*, 1727–1738. [\[CrossRef\]](#)
62. Zhu, Y.; Zabaras, N.; Koutsourelakis, P.S.; Perdikaris, P. Physics-constrained deep learning for high-dimensional surrogate modeling and uncertainty quantification without labeled data. *J. Comput. Phys.* **2019**, *394*, 56–81. [\[CrossRef\]](#)
63. Sun, L.; Gao, H.; Pan, S.; Wang, J.-X. Surrogate modeling for fluid flows based on physics-constrained deep learning without simulation data. *Comput. Methods Appl. Mech. Eng.* **2020**, *361*, 112732. [\[CrossRef\]](#)
64. Von Rueden, L.; Mayer, S.; Beckh, K.; Georgiev, B.; Giesselbach, S.; Heese, R.; Kirsch, B.; Walczak, M.; Pfrommer, J.; Pick, A.; et al. Informed machine learning—A taxonomy and survey of integrating prior knowledge into learning systems. *IEEE Trans. Knowl. Data Eng.* **2023**, *35*, 614–633. [\[CrossRef\]](#)
65. Yamazaki, Y.; Harandi, A.; Muramatsu, M.; Viardin, A.; Apel, M.; Brepols, T.; Reese, S.; Rezaei, S. A finite element-based physics-informed operator learning framework for spatiotemporal partial differential equations on arbitrary domains. *Eng. Comput.* **2025**, *41*, 1–29. [\[CrossRef\]](#)
66. Sunil, P.; Sills, R.B. Fe-pinns: Finite-element-based physics-informed neural networks for surrogate modeling. *arXiv* **2024**, arXiv:2412.07126.

67. Montavon, G.; Orr, G.; Müller, K.-R. *Neural networks: Tricks of the Trade*; Lecture Notes in Computer Science; Springer: Berlin/Heidelberg, Germany, 2012; Volume 7700.
68. Goodfellow, I.; Bengio, Y.; Courville, A.; Bengio, Y. *Deep Learning*; Adaptive Computation and Machine Learning; MIT Press: Cambridge, UK, 2016; Volume 1.
69. Liu, X.; Wang, Z.; Ji, H.; Gong, H. Application and comparison of several adaptive sampling algorithms in reduced order modeling. *Heliyon* **2024**, *10*, e34928. [[CrossRef](#)]
70. Lu, L.; Shin, Y.; Su, Y.; Karniadakis, G.E. Dying ReLU and initialization: Theory and numerical examples. *Commun. Comput. Phys.* **2020**, *28*, 1671–1706. [[CrossRef](#)]
71. Ramachandran, P.; Zoph, B.; Le, Q.V. Searching for activation functions. *arXiv* **2017**, arXiv:1710.05941.
72. Halder, R.; Stabile, G.; Rozza, G. Physics informed neural network framework for unsteady discretized reduced order system. *arXiv* **2023**, arXiv:2311.14045. [[CrossRef](#)]

Disclaimer/Publisher’s Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.